Les 4 livrables du jour, à rendre par chaque groupe

1. **Le modèle entité-association annoté** des données sensibles et des menaces (lab 1).
2. **Le schéma relationnel** : tables, clés et relations dans PostgreSQL (lab 2).
3. **Les dépendances fonctionnelles** : fermetures, clés candidates, normalisation (labs 4 et 5).
4. **Un premier script d'interrogation sécurisé** : requêtes et protection contre l'injection SQL (lab 3).

Ils sont assemblés et testés en fin de journée (section 6), puis rendus dans une seule archive.

Sommaire

Le fil de la journée

0. Préparer son poste (avant la séance)

1. Lab 1 · Modèle entité-association (1 h 30)

2. Lab 2 · Passage au modèle relationnel (1 h)

3. Lab 3 · Algèbre relationnelle et premières requêtes (1 h 30)

4. Lab 4 · Dépendances fonctionnelles et clés candidates (1 h 30)

5. Lab 5 · Formes normales (1 h 30)

6. Rendu des 4 livrables du jour

Le fil de la journée

Aujourd'hui, vous construisez la base de données de **Queen Bee**, une plateforme qui suit les ruches d'apicultrices africaines. Vous partez d'un texte, et vous finissez avec une base qui fonctionne, qui se défend et qui peut servir de preuve. Chaque lab utilise ce que le précédent a produit :

Lab	Vous partez de…	Vous produisez…	Durée
1 · Modèle entité-association	le texte du sujet	le dessin des données (MCD) et leurs menaces : `mcd.png`, `annotations.md`	1 h 30
2 · Modèle relationnel	votre MCD	les tables dans PostgreSQL : `schema.sql`, `donnees.sql`, `tests_schema.sql`	1 h
3 · Algèbre et requêtes	vos tables remplies	des requêtes, et un script qui résiste à l'injection : `requetes.sql`, `query_vuln.py`, `query.py`	1 h 30
4 · Dépendances fonctionnelles	vos requêtes	les règles cachées dans les données : `plat.sql`, `dependances.md`, `contraintes.sql`	1 h 30
5 · Formes normales	vos règles	la preuve que la base ne se contredit pas : `anomalies.sql`, `migration_3nf.sql`, `normalisation.md`	1 h 30

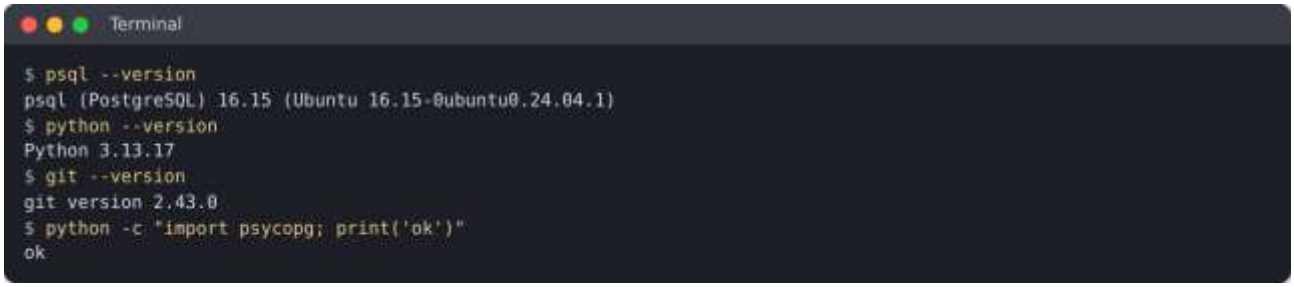
Comment lire ce TP. Chaque étape alterne deux blocs. **Exemple GreenFarm** (en vert) montre un morceau déjà fait sur une coopérative de serres connectées : vous le **lisez**, vous ne le refaites pas. **À vous** (en orange) demande le même morceau pour Queen Bee. Puis vient le morceau suivant de l'exemple, et ainsi de suite. Chaque « À vous » dit où travailler, quoi écrire, quelle commande lancer, et ce que vous devez voir. Si vous ne voyez pas le **résultat attendu**, ne passez pas à la suite : appelez l'intervenant.

0. Préparer son poste (avant la séance)

Étape 0.1 · Vérifier que les outils sont installés

Ouvrez un terminal : sous Windows, *PowerShell* (menu Démarrer › tapez « PowerShell ») ; sous macOS et Linux, *Terminal*. Tapez ces quatre commandes, une par une, en appuyant sur *Entrée* après chacune :

```
psql --version
python --version
git --version
python -c "import psycopg; print('ok')"
```



Ce que vous devez voir : quatre réponses, sans erreur (vos numéros de version peuvent être plus récents)

Étape 0.2 · À quoi sert chaque outil

Outil	C'est quoi ?	Ce que vous en ferez aujourd'hui
PostgreSQL	le serveur de base de données : il garde les tables et fait respecter les règles	héberger la base de Queen Bee
psql	la commande qui parle à PostgreSQL depuis le terminal	créer la base, lancer vos fichiers `.sql`, lire les erreurs
pgAdmin 4	l'interface graphique de PostgreSQL	regarder les tables, les données et le diagramme ; tester des requêtes
draw.io	un logiciel de diagrammes, dans le navigateur	dessiner le modèle entité-association
VS Code	l'éditeur de texte (le Bloc-notes suffirait, mais VS Code colore le code)	écrire vos fichiers `.sql`, `.py` et `.md`
Python + psycopg	un langage de script, et sa bibliothèque pour parler à PostgreSQL	écrire un petit programme qui interroge la base
Git	l'outil qui garde l'historique de vos fichiers	enregistrer chaque lab, produire l'archive à rendre

Étape 0.3 · Si une commande ne passe pas : installer

« Commande introuvable » ou « n'est pas reconnu » : l'outil manque, ou Windows ne sait pas où il est (il n'est pas dans le *Path*).

Il manque…	Installation
`psql`	postgresql.org/download : dernière version ; gardez cochés *PostgreSQL Server*, *pgAdmin 4* et *Command Line Tools* ; choisissez le mot de passe de `postgres` et **notez-le** ; laissez le port 5432. Sous Windows, ajoutez ensuite `C:\Program Files\PostgreSQL\<version>\bin` au *Path* (menu Démarrer › « variables d'environnement »), puis rouvrez le terminal. macOS : Postgres.app. Linux : `sudo apt install postgresql`.
`python`	python.org/downloads ; sous Windows, cochez *Add python.exe to PATH*. Si `python` ouvre le Microsoft Store, tapez `py` à la place.

Il manque…	Installation
`psycopg`	`pip install "psycopg[binary]"` (ou `python -m pip install "psycopg[binary]"`)
`git`	git-scm.com/downloads, options par défaut
VS Code, draw.io	code.visualstudio.com ; draw.io s'utilise en ligne : app.diagrams.net

Relancez l'étape 0.1 jusqu'à obtenir les quatre réponses.

Étape 0.4 · Créer le dossier de travail et la base

Tout votre travail de la journée vivra dans un dossier, `dossier-bdd`, et dans une base, `queenbee`. Dans le terminal :

```
mkdir dossier-bdd
cd dossier-bdd
git init
psql -U postgres -c "CREATE DATABASE queenbee"
```

`psql` demande le mot de passe de `postgres` (les caractères ne s'affichent pas, c'est normal) et répond `CREATE DATABASE`. Sous Linux, si `psql` répond `Peer authentication failed`, ajoutez `-h localhost` à toutes les commandes `psql` de la journée.

Ouvrez ensuite pgAdmin : cliquez sur *Servers › PostgreSQL*, tapez le même mot de passe ; la base `queenbee` apparaît sous *Databases*. Elle est vide : vous la remplirez au lab 2.

1. Lab 1 · Modèle entité-association (1 h 30)

Pourquoi ce lab. Avant de créer la moindre table, il faut savoir **de quoi** le système doit se souvenir et **comment** ces informations sont liées. Une erreur ici se retrouve partout ensuite. On commence donc sans ordinateur ou presque : on lit, on répond à des questions simples, puis on dessine.

Partie A · Lire le sujet et le comprendre (25 min)

> **Le sujet : Queen Bee.** L'association BeeAfrica aide des apicultrices d'Afrique de l'Ouest et du Centre. Elle suit **400 ruches** installées sur **8 sites ruchers**, dans **3 pays** : Sénégal, Côte d'Ivoire, Cameroun. Chaque ruche porte un **code** (par exemple `SN-CASA-014`), une **position GPS** et une **date d'installation**. Chaque ruche appartient à une **apicultrice** ; une apicultrice a un **nom** et un **téléphone**, elle est rattachée à un site et **adhère à une ou plusieurs coopératives**. Des **capteurs** posés dans les ruches envoient toutes les 30 minutes une **mesure** : poids de la ruche, température intérieure et extérieure, humidité, activité sonore des abeilles, niveau de batterie. Les apicultrices passent régulièrement **inspecter** leurs ruches (état de la colonie, présence de la reine, observations). Quand elles récoltent du miel, elles **déclarent la récolte** (date, quantité), puis un **coordinateur la valide**. Le système déclenche des **alertes** (essaimage, vol, capteur silencieux) avec un niveau de confiance. Quatre types d'**utilisateurs** se connectent à la plateforme : l'apicultrice, le coordinateur de site, le gestionnaire de l'association et l'administrateur. L'énoncé est volontairement incomplet : à vous de repérer ce qu'il ne dit pas.

Pour vous guider, chaque question est d'abord résolue, en bref, sur l'exemple **GreenFarm**, dont voici le sujet :

> **L'exemple : GreenFarm.** Une coopérative agricole suit les **serres** de ses **exploitations** (chaque exploitation a un nom et une région). Chaque serre a un **code** et une **surface** ; elle est gérée par un **exploitant** (nom, téléphone). Des capteurs envoient chaque heure un **relevé** (température, humidité). Les exploitants notent leurs **interventions** (arrosage, traitement). Ils **adhèrent à des groupements** de producteurs. Chacun a un **compte** pour se connecter.

Ouvrez VS Code, puis *File › Open Folder…* › `dossier-bdd`, et créez le fichier `annotations.md` (*File › New File…*). Vous y écrirez vos cinq réponses. Aucun mot technique n'est attendu : on cherche à comprendre le métier.

Exemple GreenFarm · Question 1 : de quoi le système doit-il se souvenir ?

On souligne les « choses » sur lesquelles on garde plusieurs informations : exploitation, serre, exploitant, relevé, intervention, groupement, compte. **7 choses.** La température n'en est pas une : c'est une information **sur** le relevé.

À vous · Question 1

Faites la même chose pour Queen Bee : soulignez dans le sujet, écrivez la liste et le nombre.

Exemple GreenFarm · Question 2 : que garde-t-on sur chacune, et qu'est-ce qui la distingue ?

Chose	Ce qu'on garde	Ce qui la distingue
serre	code, surface	le code
exploitant	nom, téléphone	un numéro attribué (deux exploitants s'appellent Paul Martin : le nom ne suffit pas)
relevé	heure, température, humidité	la serre et l'heure

À vous · Question 2

Recopiez le tableau, une ligne par chose de votre question 1 :

Chose	Ce qu'on garde	Ce qui la distingue
___	___	___

Exemple GreenFarm · Question 3 : comment sont-elles liées ?

Une phrase **dans chaque sens**, avec « une seule », « au plus une », « au moins une » ou « plusieurs » :

Lien	Dans un sens	Dans l'autre sens
serre – exploitant	une serre est gérée par un seul exploitant	un exploitant gère zéro, une ou plusieurs serres
exploitant – groupement	un exploitant adhère à un ou plusieurs groupements	un groupement a plusieurs adhérents
exploitant – compte	un exploitant a au plus un compte	un compte appartient au plus à un exploitant

À vous · Question 3

Lien	Dans un sens	Dans l'autre sens
ruche – apicultrice	___	___
site – ruche	___	___
apicultrice – coopérative	___	___
ruche – mesure	___	___
ruche – récolte	___	___
coordinateur – récolte validée	___	___
apicultrice – compte de connexion	___	___

Exemple GreenFarm · Question 4 : qu'est-ce que le sujet ne dit pas ?

Question au client	Notre hypothèse
Une serre peut-elle changer d'exploitant ?	oui, mais on ne garde que l'exploitant actuel
Un exploitant peut-il travailler dans deux exploitations ?	non

À vous · Question 4

Au moins 4 questions que vous poseriez au client de Queen Bee, et ce que vous supposez en attendant sa réponse (pensez au changement de site d'une ruche, et à la validation des récoltes) :

Question au client	Notre hypothèse
___	___

Exemple GreenFarm · Question 5 : qu'est-ce qui intéresserait un fraudeur ?

Le téléphone des exploitants (démarchage, hameçonnage) ; les relevés (une fausse température déclenche un arrosage inutile) ; le login (se connecter à la place d'un autre).

À vous · Question 5

Citez trois informations de Queen Bee qui intéresseraient un voleur ou un fraudeur, et ce qu'il en ferait. (Vous y reviendrez en partie C.)

Ce que vous venez de faire porte un nom

- Les « choses » de la question 1 sont des **entités** ; ce qu'on garde sur elles, leurs **attributs** ; ce qui les distingue, leur **identifiant** (on le souligne).
- Les liens de la question 3 sont des **associations**, nommées par un verbe (une apicultrice *possède* une ruche).
- Les mots « une seule », « au plus une », « au moins une », « plusieurs » deviennent des **cardinalités** : `1,1`, `0,1`, `1,n`, `0,n`. En notation Merise, on écrit la cardinalité **à côté de l'entité dont parle la phrase** : « une ruche appartient à une seule apicultrice » donne `1,1` du côté de RUCHE.
- Un lien « plusieurs » dans les deux sens (apicultrice et coopérative) est une association **n,n** ; elle peut porter sa propre information, comme la date d'adhésion.
- Les réponses de la question 4 sont les **hypothèses** du modèle : elles seront écrites dans votre rendu.

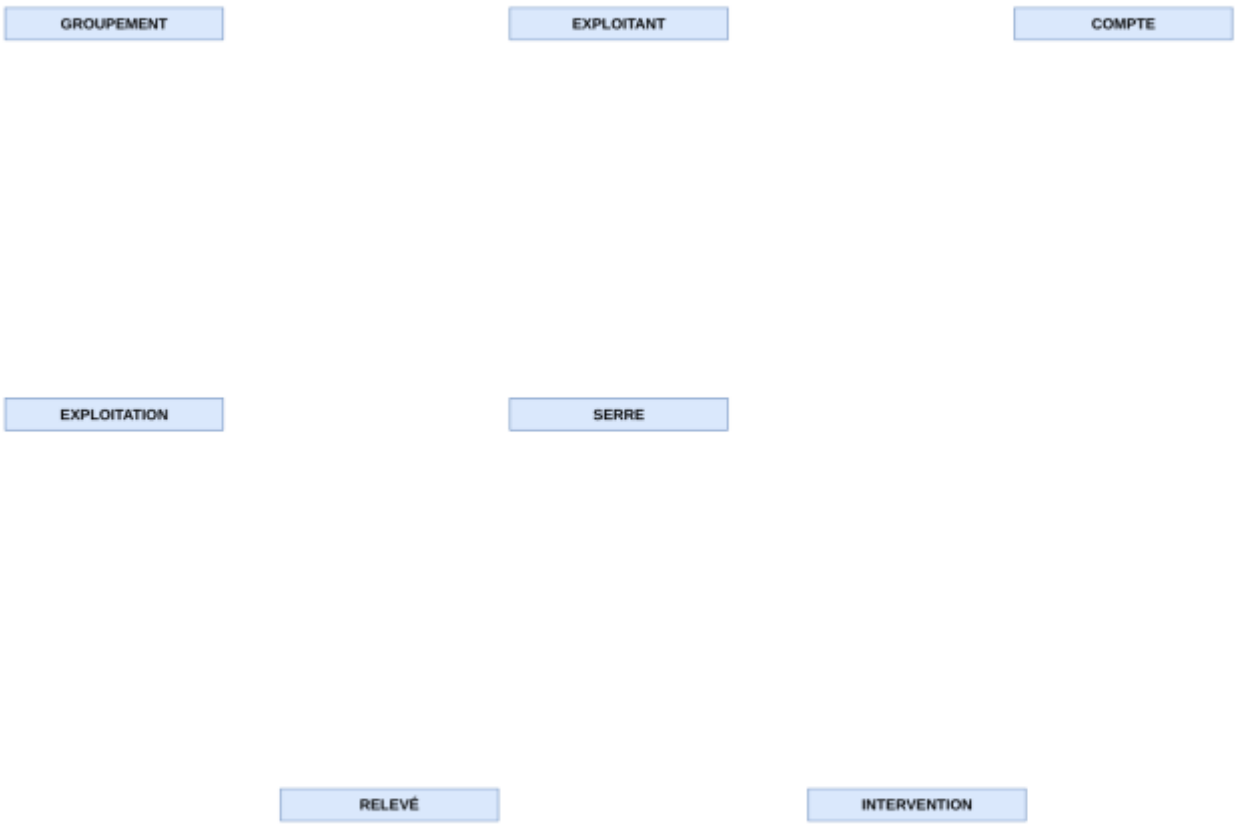
Partie B · Dessiner le MCD dans draw.io (35 min)

Vous allez transformer vos réponses en dessin, en cinq passes : les boîtes, leur contenu, les liens, les nombres, la couleur. À chaque passe, un morceau de l'exemple GreenFarm, puis le même morceau pour Queen Bee.

Ouvrez app.diagrams.net dans le navigateur, cliquez sur *Appareil*, puis *Créer un nouveau diagramme › Diagramme vide › Créer*. Les formes sont dans le panneau *Général*, à gauche.

Étape B1 · Une boîte par entité

Exemple GreenFarm



Sept entités, juste leur nom

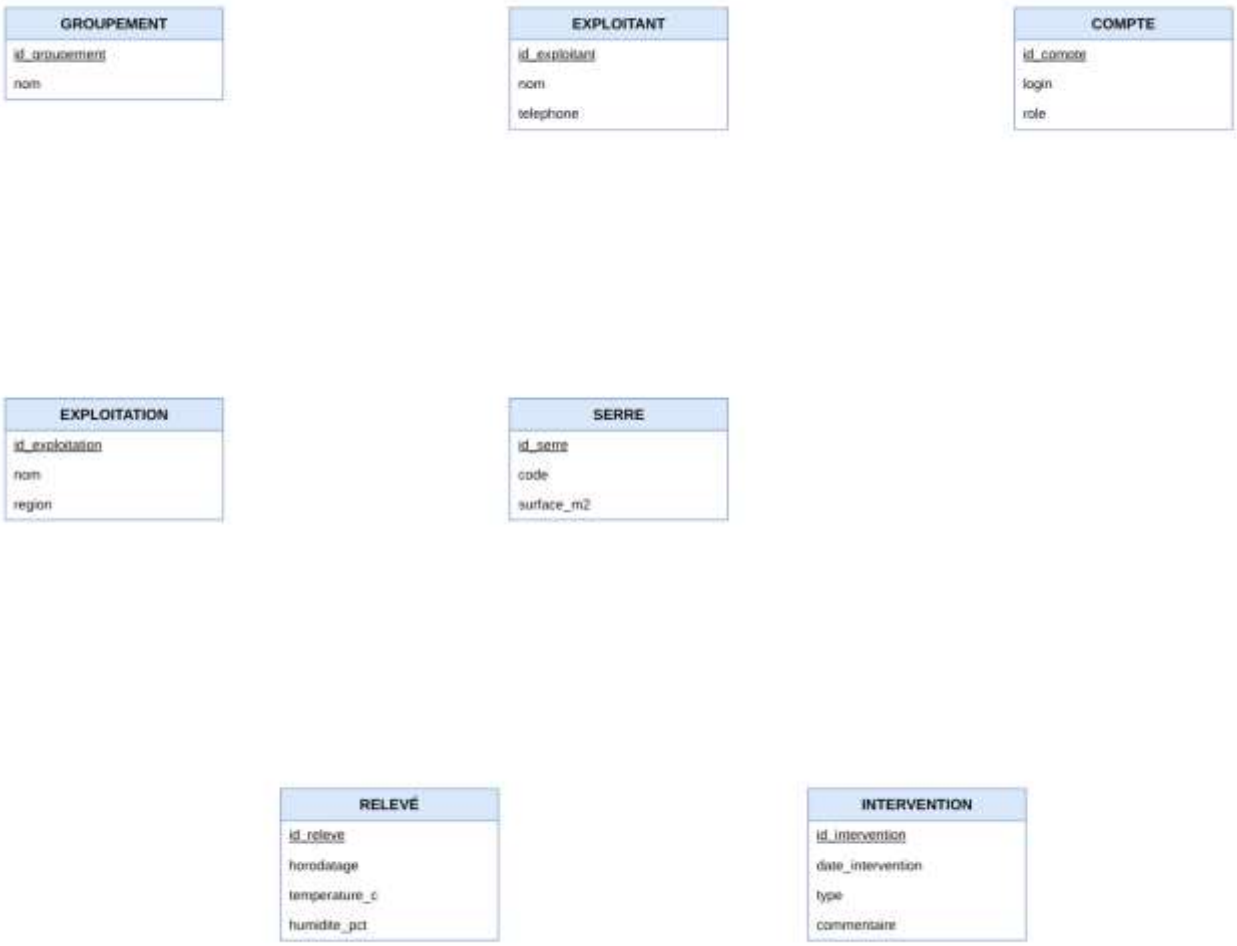
À vous

Reprenez votre réponse à la question 1. Pour chaque entité : cliquez sur la forme *Liste* (un rectangle avec un titre et trois lignes), double-cliquez sur le titre *List* et tapez le nom en majuscules (RUCHE), puis cliquez sur chaque ligne *Item* et appuyez sur *Suppr*. Laissez de l'espace entre les boîtes.

Vous devez avoir : 9 boîtes. Deux n'ont pas d'équivalent dans GreenFarm : lesquelles, et pourquoi Queen Bee en a besoin ?

Étape B2 · Les attributs, identifiant en tête

Exemple GreenFarm



Chaque entité reçoit ses attributs ; l'identifiant, souligné, en premier

GreenFarm a choisi `id_exploitant` comme identifiant, et non le nom : deux exploitants s'appellent Paul Martin.

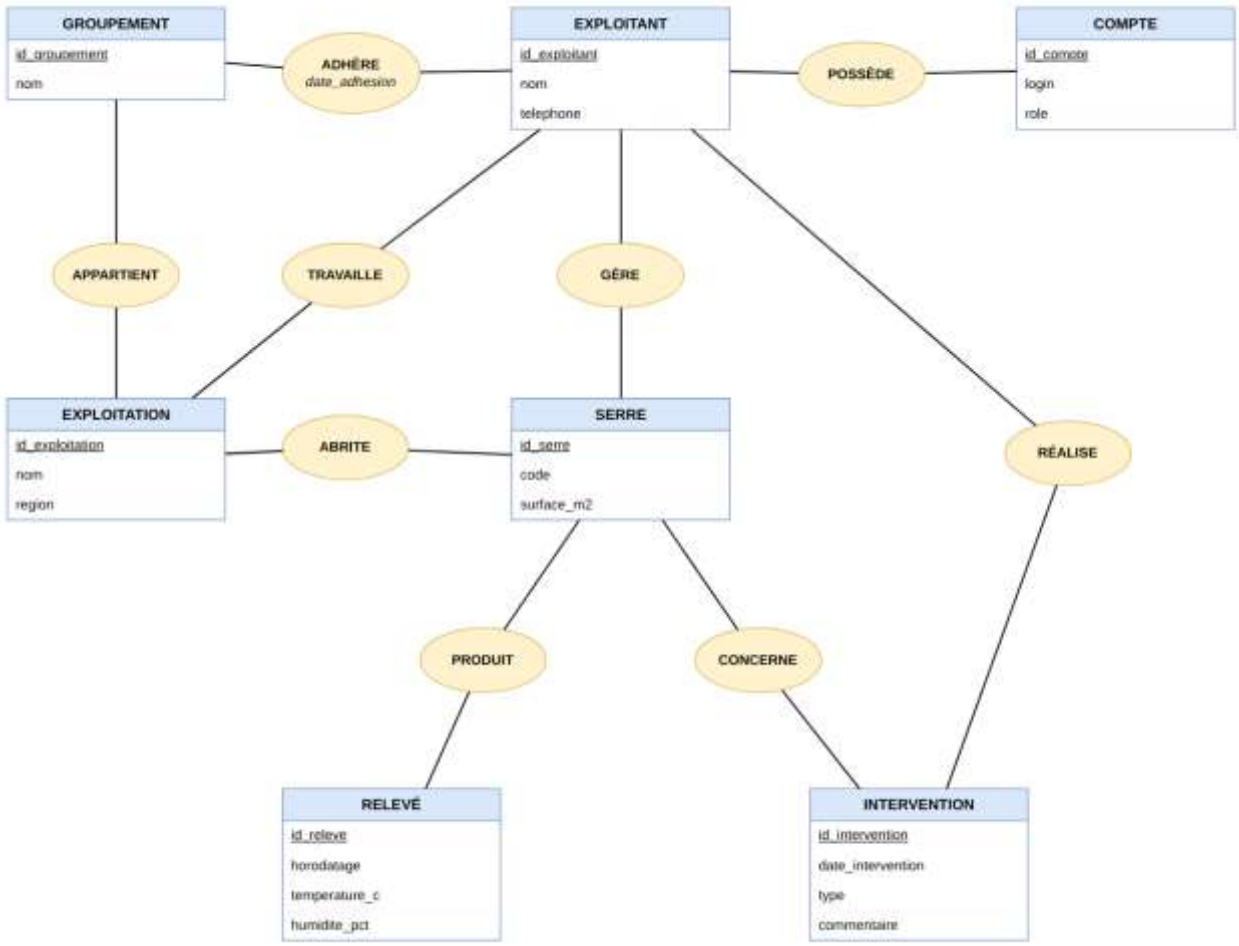
À vous

Reprenez votre tableau de la question 2. Cliquez sur la ligne du titre d'une boîte, puis *Ctrl + Entrée* : une ligne s'ajoute ; double-cliquez dessus et tapez l'attribut. L'identifiant va en première ligne, souligné (sélection du texte, *Ctrl + U*).

Vous devez avoir : chaque boîte avec son identifiant souligné.

Étape B3 · Les associations

Exemple GreenFarm



Une ellipse par lien, avec un verbe, reliée aux deux entités

L'ellipse ADHÈRE porte `date_adhesion` : cette date n'appartient ni à l'exploitant ni au groupement, mais au lien entre eux.

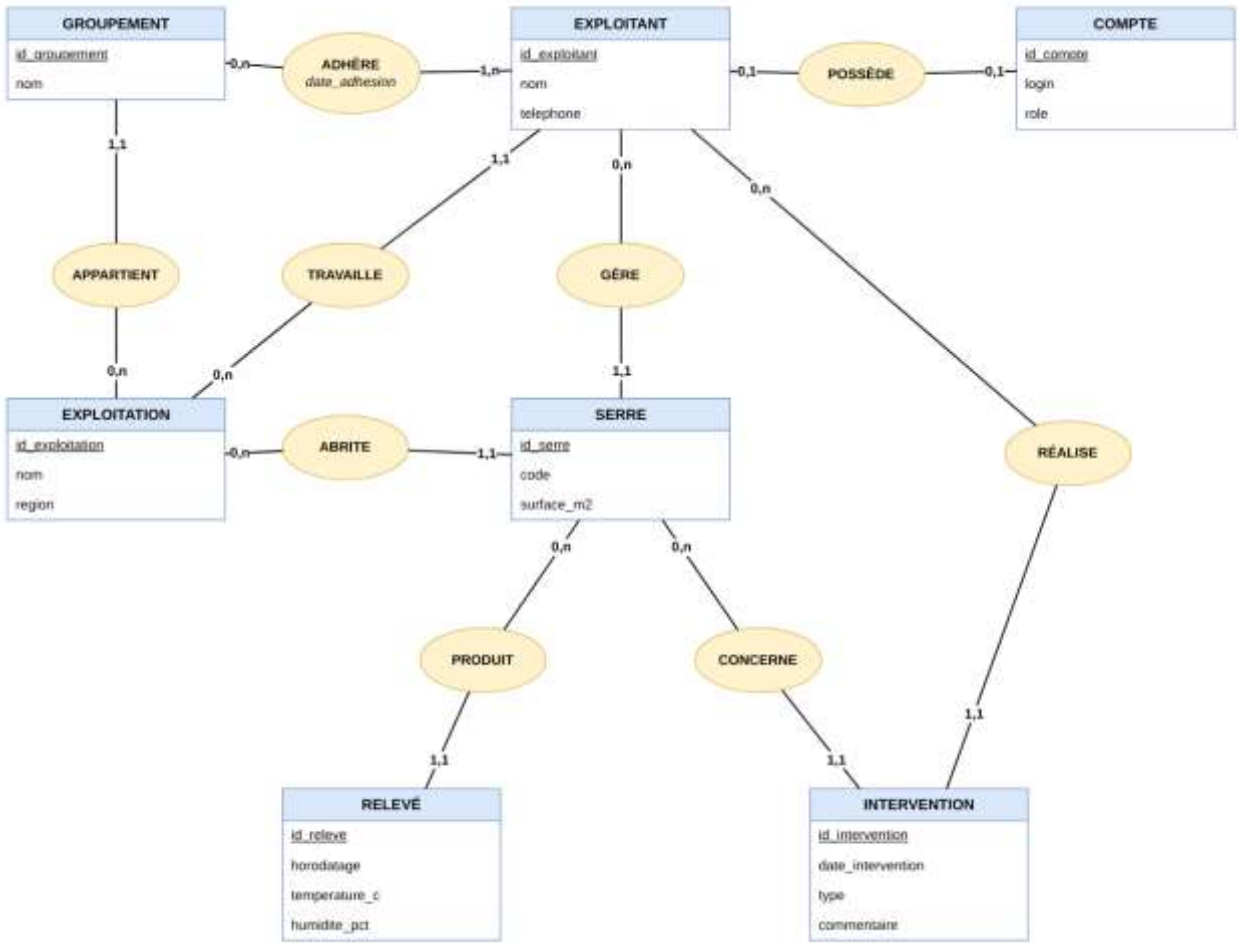
À vous

Reprenez votre tableau de la question 3. Pour chaque lien : cliquez sur l'**ellipse** (panneau *Général*) et tapez un verbe (*possède*, *émet*, *valide*…). Survolez une boîte : des flèches bleues apparaissent ; cliquez sur l'une d'elles et glissez jusqu'à l'ellipse ; faites de même depuis la deuxième boîte. Pour enlever la pointe d'un trait : cliquez dessus, puis dans le panneau *Style*, à droite, réglez les deux extrémités sur *Aucune*.

Vous devez avoir : une ellipse par lien, sans flèche ; une ellipse qui porte une date ; une ellipse entre le compte du coordinateur et la récolte.

Étape B4 · Les cardinalités

Exemple GreenFarm



Sur chaque trait, la cardinalité près de l'entité qu'elle décrit

Lecture : près de SERRE, `1,1` veut dire « une serre est gérée par exactement un exploitant » ; près d'EXPLOITANT, `0,n` veut dire « un exploitant gère zéro, une ou plusieurs serres ».

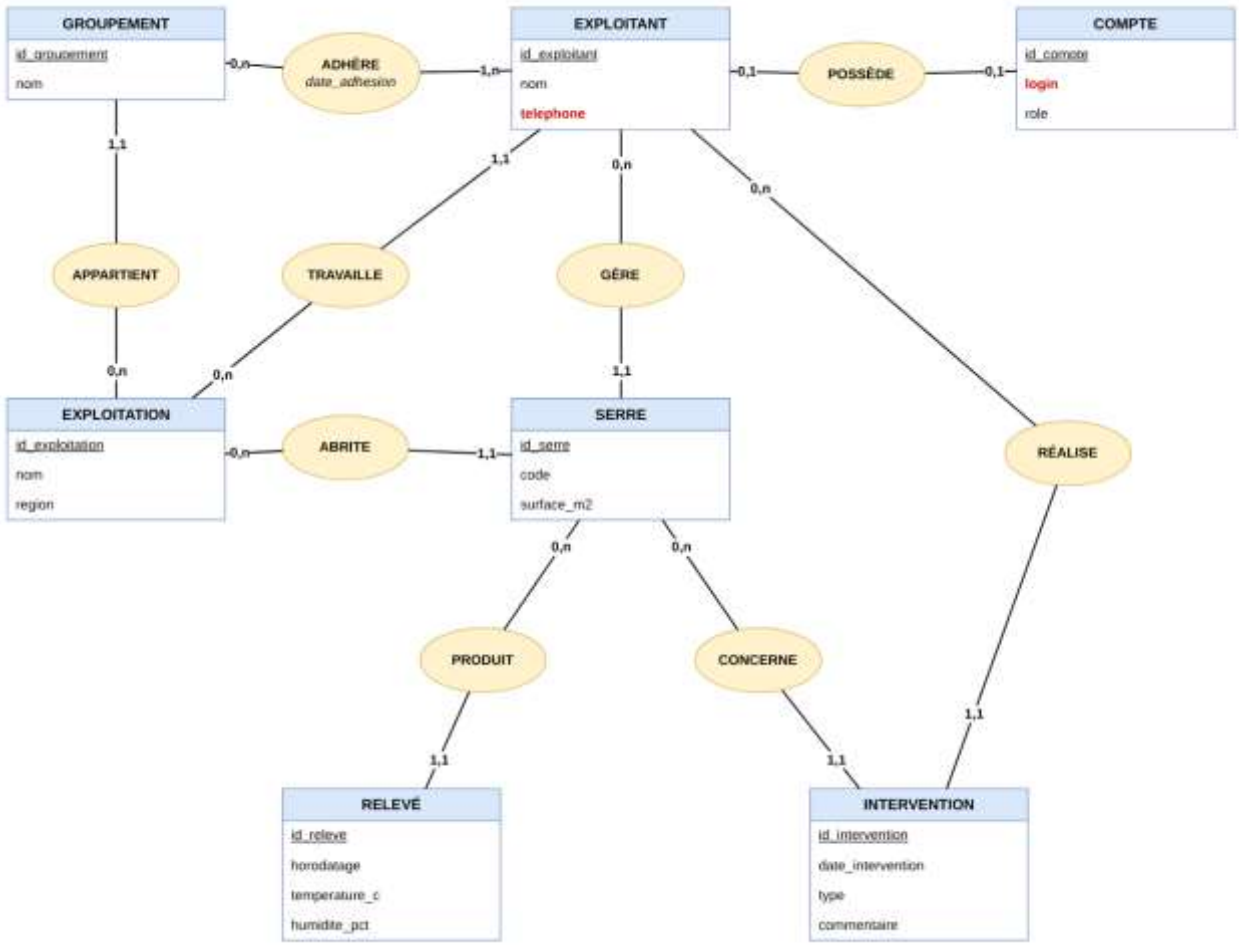
À vous

Traduisez chaque phrase de la question 3 : « une seule » → `1,1` ; « au plus une » → `0,1` ; « au moins une » → `1,n` ; « zéro, une ou plusieurs » → `0,n`. Double-cliquez sur le trait **près de la boîte dont parle la phrase** et tapez la cardinalité.

Vous devez avoir : deux cardinalités par trait. Un seul lien a `n` des deux côtés : celui qui porte la date.

Étape B5 · Les données sensibles, puis l'enregistrement

Exemple GreenFarm



Le téléphone et le login, sensibles, sont en rouge

À vous

Reprenez votre réponse à la question 5. Pour chaque attribut cité : cliquez sur sa ligne, puis dans le panneau *Texte*, à droite, choisissez une police rouge. Enregistrez ensuite deux fois, dans `dossier-bdd` :

1. *Fichier › Enregistrer sous…* : `mcd.drawio` (le fichier modifiable) ;
2. *Fichier › Exporter en tant que › PNG… › Exporter* : `mcd.png` (l'image à rendre).

> **Résultat attendu :** 9 entités avec leur identifiant souligné, une ellipse par lien, deux cardinalités par trait, ADHÈRE avec sa date, le lien de validation entre UTILISATEUR et RÉCOLTE, les attributs sensibles en rouge.

Partie C · Annoter la sensibilité et les menaces (30 min)

Le dessin dit **ce que** la base contient. Il reste à dire **qui** a le droit d'y toucher, et **ce qui se passe** si quelqu'un triche.

STRIDE appliqué aux données

Pour chaque entité, demandez-vous ce que gagne un attaquant à la **lire**, à la **modifier** ou à la **supprimer**. Notez la menace avec une lettre : **S** usurper une identité, **T** altérer, **R** nier avoir agi, **I** divulguer, **D** rendre indisponible, **E** obtenir plus de droits. Notez aussi qui doit pouvoir lire et modifier : chacun n'accède qu'à ce dont il a besoin (**moindre privilège**).

Exemple GreenFarm

Entité	Sensibilité	Qui lit	Qui modifie	Menace
EXPLOITANT	confidentielle (le téléphone est une donnée personnelle)	admin, l'exploitant	admin	I : démarchage, hameçonnage
RELEVÉ	interne	tous	les capteurs	T : une fausse température déclenche un arrosage inutile
COMPTE	critique	admin	admin	S : se connecter à la place d'un autre

À vous

Dans `annotations.md`, recopiez ce tableau avec **une ligne par entité** de votre MCD. Utilisez les quatre rôles du sujet (apicultrice, coordinateur, gestionnaire, administrateur) et **au moins 4 lettres STRIDE différentes**. Trois cas doivent y figurer : une mesure rejouée, une récolte validée sans qu'on sache par qui, l'effacement de l'historique des actions.

Entité	Sensibilité (publique, interne, confidentielle, critique)	Qui lit	Qui modifie	Menace STRIDE et ce que gagne l'attaquant
RUCHE	___	___	___	I : connaître la position GPS pour voler la ruche
___	___	___	___	___

Enregistrez le lab. Dans le terminal, depuis `dossier-bdd` :

```
git add annotations.md mcd.png mcd.drawio
git commit -m "Lab 1 : MCD Queen Bee annoté"
```

`git add` choisit les fichiers à enregistrer ; `git commit` les enregistre dans l'historique, avec un message.

Testez-vous · Lab 1

1. La date d'adhésion appartient-elle à l'apicultrice, à la coopérative, ou au lien entre les deux ?
2. Pourquoi le nom est-il un mauvais identifiant ?
3. Quelle lettre STRIDE correspond au rejeu d'une ancienne mesure de poids ?
4. « Une ruche appartient à une seule apicultrice » : quelle cardinalité, et de quel côté ?

2. Lab 2 · Passage au modèle relationnel (1 h)

Pourquoi ce lab. Votre MCD est un dessin : PostgreSQL ne sait pas le lire. Il faut le traduire en **tables** (des tableaux à colonnes fixes, une ligne par élément), les écrire dans un fichier, les créer, les remplir, puis vérifier que la base refuse les données absurdes.

Étape 2.1 · Du MCD aux tables, règle par règle (15 min)

Quatre règles suffisent. Pour chacune : ce qu'en fait GreenFarm, puis ce que vous en faites. Écrivez vos réponses dans `annotations.md`, sous un titre `## Schéma relationnel`.

Exemple GreenFarm · Règle 1 : une entité devient une table

Chaque entité devient une table ; son identifiant devient la **clé primaire** (jamais vide, jamais en double). GreenFarm obtient 7 tables :

```
exploitation (id, …)   groupement (id, …)   exploitant (id, …)   serre (id, …)
releve (id, …)         intervention (id, …) compte (id, …)
```

À vous · Règle 1

Commencez ce tableau, une ligne par entité de votre MCD ; laissez la dernière colonne vide pour l'instant :

Table	Clé primaire	Autres colonnes	Clés étrangères (vers quelle table)
ruche	id	code, latitude, longitude, date_installation	
___	___	___	

Pour chaque clé primaire, choisissez : un code lisible (le site `SN-CASA` : court, stable, pas secret) ou un **UUID**, un identifiant aléatoire généré par la base (pour tout ce qu'une application exposera : un numéro 1, 2, 3 **se devine**, qui voit `/ruches/42` essaie `/ruches/43`).

Exemple GreenFarm · Règle 2 : un lien 1,1 – 0,n devient une clé étrangère

« Une serre est gérée par un seul exploitant » (`1,1` côté SERRE). La serre garde donc l'identifiant de son exploitant dans une colonne `exploitant_id` : c'est une **clé étrangère**, placée du côté **1,1**.

```
serre (id, code, surface_m2, #exploitation_id, #exploitant_id)
```

À vous · Règle 2

Pour chaque lien 1,1 – 0,n de votre MCD, ajoutez la clé étrangère dans la dernière colonne, du côté 1,1 : `site_id (site), apicultrice_id (apicultrice)` pour la ruche, et ainsi de suite.

Exemple GreenFarm · Règle 3 : un lien n,n devient une table

ADHÈRE a `n` des deux côtés : aucune des deux tables ne peut garder « la » clé de l'autre. On crée une table de plus, dont la clé primaire réunit les deux clés étrangères, et qui garde l'attribut du lien :

```
adhesion (#exploitant_id, #groupement_id, date_adhesion)
```

À vous · Règle 3

Quel lien de votre MCD devient une table en plus ? Ajoutez-la au tableau.

Exemple GreenFarm · Règle 4 : un lien 0,1 – 0,1 devient une clé étrangère unique

Un exploitant a au plus un compte, un compte appartient au plus à un exploitant : la clé étrangère va dans `compte`, avec `UNIQUE` pour qu'un exploitant n'ait jamais deux comptes.

```
compte (id, login, role, #exploitant_id UNIQUE)
```

À vous · Règle 4

Appliquez la même règle au lien entre l'apicultrice et son compte. Puis comptez : combien de tables, combien de clés étrangères ?

Pour vérifier : avec le MCD attendu, **10 tables** et **14 clés étrangères**. Nommage : minuscules, singulier, sans accent (`date_installation`) ; une clé étrangère porte le nom de la table visée suivi de `_id`.

Étape 2.2 · Créer les tables dans un fichier SQL (20 min)

Votre tableau va devenir du **SQL**, le langage de PostgreSQL, écrit dans un fichier `schema.sql`. Un fichier se relit, se corrige, se relance et se rend. Vous allez l'écrire table par table, et le lancer dès la première.

Exemple GreenFarm · La première table, et comment lancer le fichier

Le fichier commence par supprimer les tables (pour pouvoir le relancer autant de fois qu'on veut), puis crée la table qui ne pointe vers aucune autre. Chaque ligne entre parenthèses est une colonne : son nom, son type, ses règles.

```
-- schema.sql : base GreenFarm
DROP TABLE IF EXISTS intervention, releve, serre, adhesion, compte,
                     exploitant, groupement, exploitation CASCADE;

CREATE TABLE exploitation (
  id     text PRIMARY KEY,            -- clé primaire : un code lisible, 'FR-BEAU'
  nom    text NOT NULL,               -- texte obligatoire
  region text NOT NULL
);
```

On ouvre un terminal **dans le dossier du fichier** (dans VS Code : *Terminal › New Terminal* ; ou le terminal du PC, puis `cd dossier-bdd`) et on lance :

```
psql -U postgres -d greenfarm -v ON_ERROR_STOP=1 -f schema.sql
```

Mot à mot : `psql` se connecte en tant que `postgres` (`-U`) à la base `greenfarm` (`-d`), exécute le fichier `schema.sql` (`-f`) et s'arrête à la première erreur (`-v ON_ERROR_STOP=1`). Il répond `CREATE TABLE`.

À vous · La première table

1. **Dans VS Code**, créez `schema.sql` dans `dossier-bdd`. Écrivez le `DROP TABLE IF EXISTS … CASCADE;` avec toutes les tables de votre tableau, puis le `CREATE TABLE` du site.
2. **Enregistrez** (*Ctrl + S*), puis **dans le terminal**, depuis `dossier-bdd` :

```
psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f schema.sql
psql -U postgres -d queenbee -c "\dt"
```

Vous devez voir : `CREATE TABLE`, puis `\dt` (la liste des tables) affiche `site`. Des lignes `NOTICE: table … does not exist, skipping` apparaissent : le `DROP` n'avait rien à supprimer, c'est normal.

Exemple GreenFarm · Une table qui pointe vers une autre

L'exploitant pointe vers son exploitation. `uuid … DEFAULT gen_random_uuid()` génère l'identifiant tout seul ; `REFERENCES` crée la clé étrangère. La table visée doit **déjà exister** : c'est pourquoi `exploitation` est écrite avant.

```
CREATE TABLE exploitant (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nom             text NOT NULL,
  telephone       text NOT NULL,
  exploitation_id text NOT NULL REFERENCES exploitation(id)   -- clé étrangère
);
```

À vous · Les tables qui pointent vers d'autres

1. À la suite du fichier, écrivez la coopérative, l'apicultrice et la ruche. **Donnez un nom à chaque clé étrangère**, sous la forme `table_colonne_fk` : les messages d'erreur seront lisibles, et le jour 2 utilisera ces noms.

```
  apicultrice_id uuid NOT NULL CONSTRAINT ruche_apicultrice_fk REFERENCES apicultrice(id)
```

2. Types utiles : `text`, `uuid`, `date`, `numeric(8,5)` (nombre à virgule, pour le GPS), `timestamptz` (date et heure), `smallint`, `boolean`, `bigint`.
3. Une valeur ne doit jamais se répéter dans les ruches : laquelle ? Ajoutez `UNIQUE` après son type.
4. **Enregistrez** et relancez la même commande (flèche haut, *Entrée*). **Vous devez voir :** un `CREATE TABLE` par table.

Si une `ERROR` s'affiche, lisez-la : `relation "site" does not exist` → la table visée n'est pas encore créée (changez l'ordre) ; `syntax error at or near …` → la faute est juste avant le mot cité (souvent une virgule oubliée, ou en trop avant la parenthèse fermante). Corrigez, enregistrez, relancez.

Exemple GreenFarm · La table d'association

```
CREATE TABLE adhesion (
  exploitant_id uuid NOT NULL REFERENCES exploitant(id),
  groupement_id uuid NOT NULL REFERENCES groupement(id),
  date_adhesion date NOT NULL DEFAULT current_date,   -- la date du jour par défaut
  PRIMARY KEY (exploitant_id, groupement_id)          -- clé primaire sur deux colonnes
);
```

À vous · L'adhésion et les comptes

Écrivez la table d'adhésion, puis celle des comptes (rôle, login, site, apicultrice). Une apicultrice ne doit jamais avoir deux comptes : où mettez-vous `UNIQUE` ? (Le login unique attendra le lab 4.) Enregistrez, relancez.

Exemple GreenFarm · Des règles qui protègent les données

Le capteur d'une serre envoie un relevé par heure. Si un attaquant renvoie un ancien relevé, la base doit le refuser : le couple (serre, horodatage) ne doit jamais se répéter.

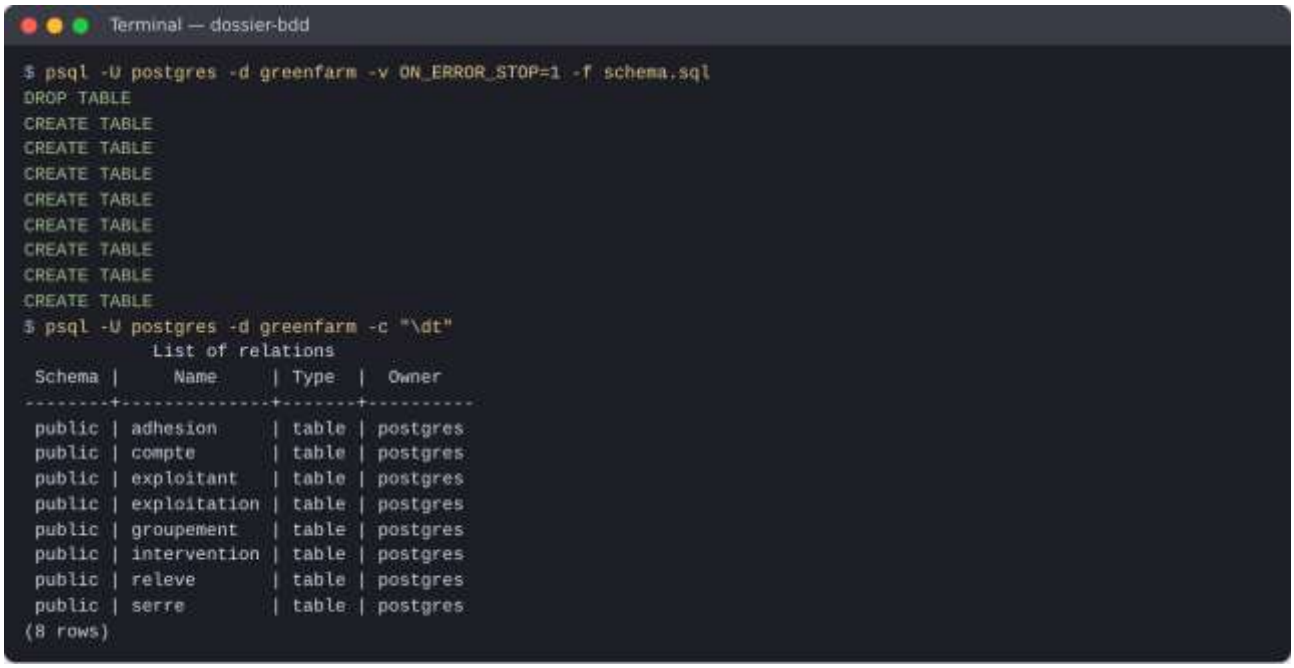
```
CREATE TABLE releve (
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  serre_id      uuid NOT NULL REFERENCES serre(id),
  horodatage    timestamptz NOT NULL,
  temperature_c numeric(4,1),
  humidite_pct  smallint,
  UNIQUE (serre_id, horodatage)        -- anti-rejeu
);
```

À vous · Les mesures, inspections, récoltes et alertes

Écrivez les quatre dernières tables. Pour chacune, une question de sécurité :

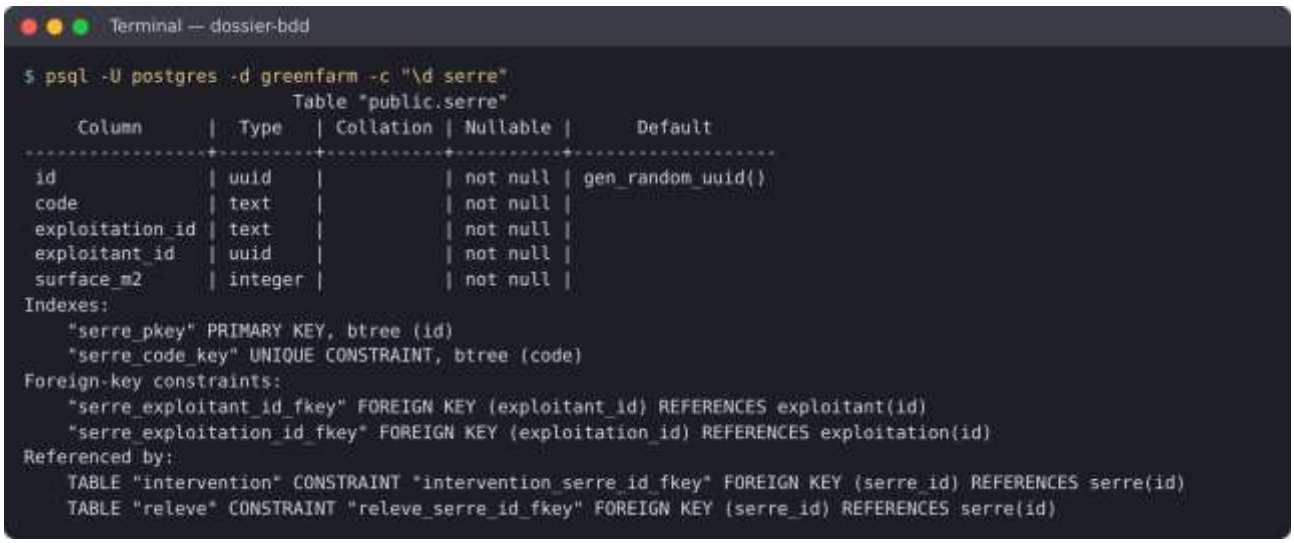
- les mesures : quel couple de colonnes ne doit jamais se répéter ? → `UNIQUE (…, …)`
- les récoltes : un volume peut-il être négatif ? une récolte peut-elle être « validée » sans qu'on sache **par qui** et **quand** ? → deux `CHECK (…)` ; et `validee_par` est une clé étrangère vers les comptes
- les alertes : entre quelles valeurs est le niveau de confiance ? → `CHECK (… BETWEEN … AND …)`

Exemple GreenFarm · Le fichier complet, lancé



schema.sql lancé, puis \dt affiche les 8 tables de GreenFarm

`\d serre` montre le détail d'une table : colonnes, clé primaire, `UNIQUE`, clés étrangères.



\d serre

À vous · Vérifier votre schéma

Enregistrez, puis **dans le terminal** :

```
psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f schema.sql
psql -U postgres -d queenbee -c "\dt"
psql -U postgres -d queenbee -c "\d ruche"
```

> **Résultat attendu :** aucune `ERROR` ; `\dt` affiche vos 10 tables ; `\d ruche` montre la clé primaire, le `UNIQUE` sur le code et les deux clés étrangères en `_fk`.

Étape 2.3 · Remplir les tables (20 min)

Des tables vides ne prouvent rien. On écrit un **jeu d'essai** dans `donnees.sql` : quelques données choisies pour que les labs suivants aient quelque chose à montrer. Ses conditions :

Données	Conditions
sites	4, dans les 3 pays, dont 2 au Sénégal (écrit `Sénégal`, avec l'accent)
coopératives	1 par site ; chaque apicultrice adhère à celle de son site
apicultrices	8, dont **deux homonymes** (deux « Aminata Diop » sur deux sites) et une « Awa Diop »
ruches	11, dont `SN-CASA-099`, et au moins une apicultrice qui en a plusieurs
mesures	une toutes les 30 minutes pendant 24 h, pour chaque ruche
comptes	un par apicultrice (login « initiale.nom », donc `a.diop` en triple), plus un coordinateur et un administrateur
inspections	6, avec au moins **2 ruches jamais inspectées**
récoltes	3, dont une déclarée et une validée (avec le validateur et la date)
alertes	4, dont 2 ouvertes, sur 2 sites

Exemple GreenFarm · Écrire les lignes une par une

Le fichier est encadré par `BEGIN;` et `COMMIT;` : tout passe, ou rien ne passe. Les tables se remplissent **dans l'ordre de leur création**. On n'écrit pas l'`id` : la base le génère.

```
-- donnees.sql : jeu d'essai GreenFarm
BEGIN;
INSERT INTO exploitation (id, nom, region) VALUES
  ('FR-BEAU', 'Beauce',   'Centre-Val de Loire'),
  ('FR-PROV', 'Provence', 'PACA');
INSERT INTO exploitant (nom, telephone, exploitation_id) VALUES
  ('Paul Martin', '06 11 22 33 44', 'FR-BEAU'),
  ('Paul Martin', '06 99 88 77 66', 'FR-PROV');      -- homonyme volontaire
```

À vous · Sites, coopératives, apicultrices

Dans VS Code, créez `donnees.sql`, commencez par `BEGIN;`, puis écrivez les `INSERT … VALUES` des sites, des coopératives et des apicultrices, selon les conditions.

Exemple GreenFarm · Retrouver l'identifiant d'une autre table

La serre doit contenir l'UUID de son exploitant. On ne le recopie jamais à la main : une **sous-requête** le retrouve à partir d'une information unique, le téléphone.

```
INSERT INTO serre (code, exploitation_id, exploitant_id, surface_m2) VALUES
  ('FR-BEAU-01', 'FR-BEAU', (SELECT id FROM exploitant WHERE telephone = '06 11 22 33 44'), 400);
```

À vous · Les ruches

Écrivez les 11 ruches, avec une sous-requête sur le téléphone de leur apicultrice.

Exemple GreenFarm · Laisser la base calculer les lignes

Quand les lignes se déduisent d'autres tables, `INSERT … SELECT` les calcule. `generate_series(0, 23)` produit les nombres 0 à 23 : une ligne par heure.

```
-- chaque exploitant adhère au groupement de son exploitation
INSERT INTO adhesion (exploitant_id, groupement_id)
SELECT e.id, g.id FROM exploitant e JOIN groupement g ON g.exploitation_id = e.exploitation_id;

-- 24 relevés par serre
INSERT INTO releve (serre_id, horodatage, temperature_c, humidite_pct)
SELECT s.id, TIMESTAMPTZ '2027-03-14 00:00+00' + h * INTERVAL '1 hour',
       round((18 + random() * 10)::numeric, 1), 60 + floor(random() * 20)::int
FROM serre s CROSS JOIN generate_series(0, 23) AS h;
COMMIT;
```



Lancé : un INSERT par commande, avec le nombre de lignes ajoutées ; puis le comptage des relevés

À vous · Adhésions, comptes, mesures, puis le reste

1. Écrivez les adhésions, les comptes et les mesures avec `INSERT … SELECT`. Login : `lower(left(nom, 1)) || '.' || lower(split_part(nom, ' ', 2))` ; mesures : `generate_series(0, 47)` et `INTERVAL '30 minutes'`.
2. Ajoutez les inspections, les récoltes et les alertes, puis `COMMIT;`.
3. **Enregistrez**, puis **dans le terminal**, lancez le fichier et comptez :

```
psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f donnees.sql
psql -U postgres -d queenbee -c "SELECT (SELECT count(*) FROM ruche) AS ruches, (SELECT count(*) FROM
mesure) AS mesures, (SELECT count(*) FROM adhesion) AS adhesions, (SELECT count(*) FROM utilisateur) AS
comptes"
```

> **Résultat attendu :** 11 ruches, 528 mesures (11 × 48), 8 adhésions, 10 comptes (adaptez les noms de tables aux vôtres).

Si une commande échoue, rien n'est enregistré (effet de `BEGIN` / `COMMIT`). Corrigez, puis relancez `schema.sql` (tables vides) **et** `donnees.sql`.

Question : relancez `donnees.sql` une seconde fois, sans `schema.sql`. Que se passe-t-il, et quelle règle de votre schéma vous protège ?

Étape 2.4 · Regarder la base dans pgAdmin (10 min)

Jusqu'ici, vous n'avez vu la base qu'à travers le terminal. pgAdmin permet de la **voir**.

Exemple GreenFarm · Les tables



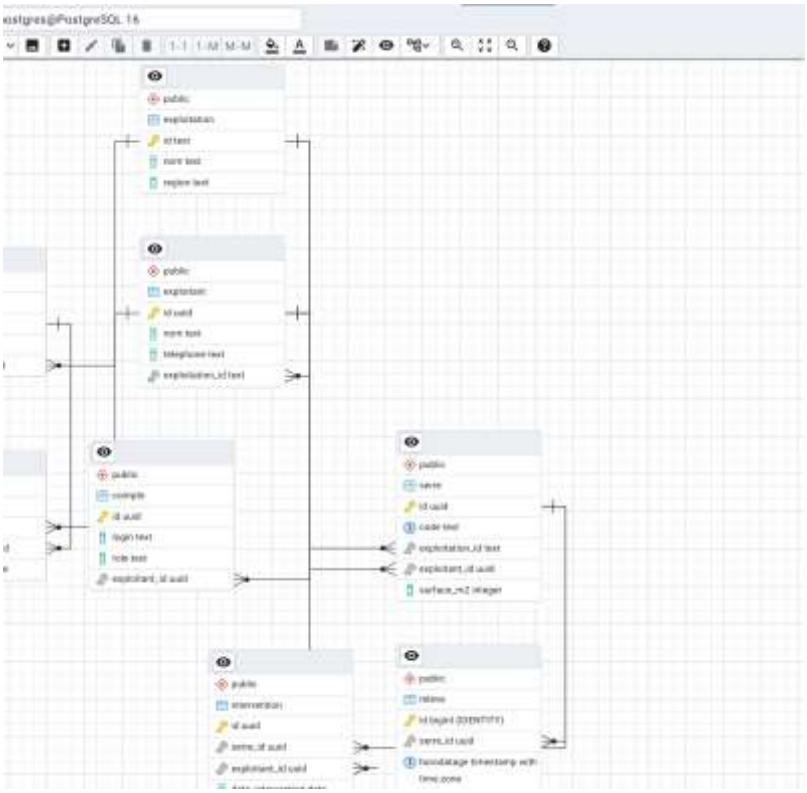
Les tables de GreenFarm dans l'arbre de pgAdmin

À vous · Les tables et les données

Dans pgAdmin, déroulez *Databases › queenbee › Schemas › public › Tables*, puis clic droit › *Refresh* : vos 10 tables apparaissent. Clic droit sur la table des ruches › *View/Edit Data › All Rows* : les 11 ruches, avec leurs UUID.

Exemple GreenFarm · Le diagramme

Clic droit sur la base › *ERD For Database* : pgAdmin dessine les tables et un trait par `REFERENCES`. On doit retrouver le MCD.



Le diagramme dessiné par pgAdmin à partir des clés étrangères

À vous · Le diagramme et le Query Tool

1. Clic droit sur `queenbee` › *ERD For Database*. Chaque association de votre MCD doit être un trait, ou une table pour ADHÈRE. Sinon, corrigez `schema.sql` et relancez les deux fichiers.
2. Ouvrez le **Query Tool** (clic droit sur `queenbee` › *Query Tool*) : on y tape une requête et on l'exécute avec *F5*. Vous vous en servirez souvent. Exécutez :

```
SELECT conrelid::regclass AS table_, confrelid::regclass AS pointe_vers, conname
FROM pg_constraint WHERE contype = 'f' ORDER BY 1, 2;
```

Résultat attendu : une ligne par clé étrangère, avec vos noms en `_fk` (14 avec le MCD attendu).

Étape 2.5 · Essayer de casser la base (10 min)

Une règle n'a de valeur que si elle **refuse** vraiment. Vous jouez l'attaquant, dans un fichier `tests_schema.sql` : pour chaque test, la commande, puis en commentaire l'attendu et l'obtenu.

Exemple GreenFarm · Une donnée orpheline

```
-- Test : une serre dont l'exploitant n'existe pas. Attendu : refus
INSERT INTO serre (code, exploitation_id, exploitant_id, surface_m2)
VALUES ('FR-BEAU-99', 'FR-BEAU', gen_random_uuid(), 100);
```

```
psql — greenfarm

greenfarm=# -- une serre dont l'exploitant n'existe pas
greenfarm-# INSERT INTO serre (code, exploitation_id, exploitant_id, surface_m2)
greenfarm-# VALUES ('FR-BEAU-99', 'FR-BEAU', gen_random_uuid(), 100);
ERROR:  insert or update on table "serre" violates foreign key constraint "serre_exploitant_id_fkey"
DETAIL:  Key (exploitant_id)=(b7f71986-1b78-4db2-902d-2c2a39117608) is not present in table "exploitant".
```

Refusée par la clé étrangère

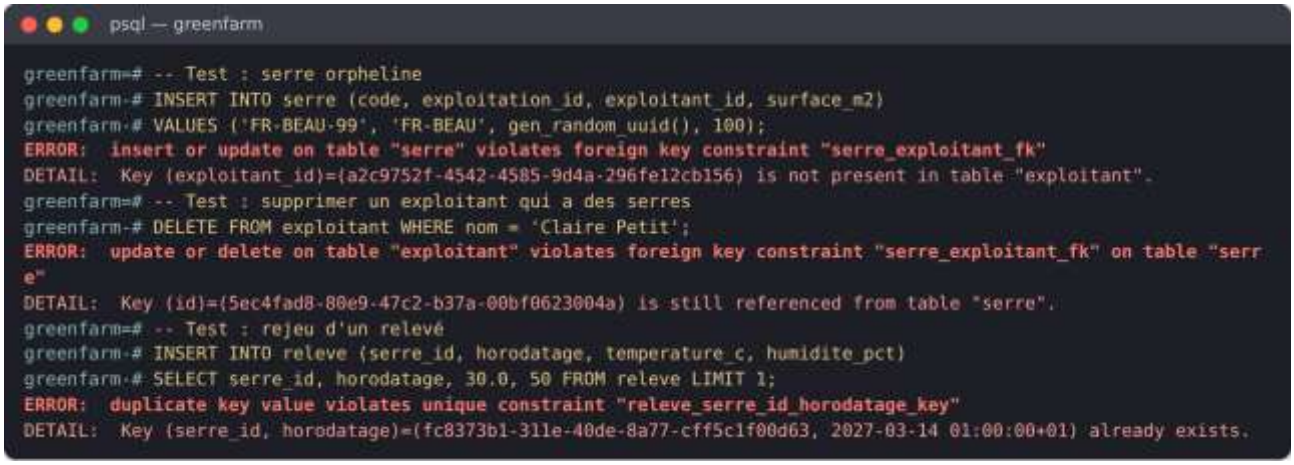
À vous · Tests 1 à 3 : les clés étrangères et NOT NULL

Dans VS Code, créez `tests_schema.sql` :

#	Test	Attendu
1	une ruche dont l'apicultrice n'existe pas (`gen_random_uuid()`)	refus
2	supprimer une apicultrice qui a des ruches	refus
3	une ruche sans apicultrice	refus

Exemple GreenFarm · Le rejeu

Rejouer un relevé déjà reçu (même serre, même heure) est refusé par l'`UNIQUE` anti-rejeu, comme la suppression d'un exploitant qui a des serres :



Les trois refus de GreenFarm

À vous · Tests 4 à 8, puis lancement

#	Test	Attendu
4	une table **sans** clé étrangère : y mettre une ruche orpheline, puis la joindre aux apicultrices (commencez par `DROP TABLE IF EXISTS`, finissez par `DROP TABLE`)	accepté, puis 0 ligne : l'orpheline devient invisible
5	une table avec `id serial` : y insérer 3 lignes, les afficher	1, 2, 3 : devinable
6	une mesure déjà reçue (même ruche, même instant)	refus
7	les ruches dont le site diffère de celui de leur apicultrice	selon votre hypothèse du lab 1
8	passer une récolte « déclarée » à « validée » sans validateur	refus

Enregistrez, puis **dans le terminal**, lancez le fichier **sans** `ON_ERROR_STOP` : les erreurs sont voulues, `psql` doit continuer après chacune.

```
psql -U postgres -d queenbee -f tests_schema.sql
```

> **Résultat attendu :** une `ERROR` pour les tests 1, 2, 3, 6 et 8, chacune nommant la règle qui refuse. Collez chaque message sous son test, puis répondez en commentaire, à la fin : un UUID suffit-il à protéger la ruche d'une autre apicultrice ?

Enregistrez le lab :

```
git add annotations.md schema.sql donnees.sql tests_schema.sql
git commit -m "Lab 2 : schéma relationnel"
```

Testez-vous · Lab 2

1. Dans quelle table met-on la clé étrangère d'une association 1,1 – 0,n ?
2. Pourquoi la clé primaire d'une table d'association est-elle composée de deux colonnes ?
3. Que protège une contrainte unique sur le couple (ruche, horodatage) ?
4. Pourquoi écrit-on la table des sites avant celle des ruches dans `schema.sql` ?

3. Lab 3 · Algèbre relationnelle et premières requêtes (1 h 30)

Pourquoi ce lab. La base est remplie : on va lui **poser des questions**. On les formule d'abord en algèbre relationnelle (quelques opérations seulement), puis en SQL. Ensuite, on regarde ce qui arrive quand un programme laisse l'utilisateur écrire dans la requête.

Étape 3.1 · Cinq questions à la base (45 min)

Dans VS Code, créez `requetes.sql`. Pour chaque question : l'algèbre en commentaire, la requête, puis le nombre de lignes obtenu. Mettez chaque requête au point dans le **Query Tool** (*F5*), puis recopiez-la dans le fichier.

Exemple GreenFarm · Garder des lignes et des colonnes

La **sélection** σ garde des lignes (`WHERE`) ; la **projection** π garde des colonnes (`SELECT col1, col2`). La projection protège aussi : on n'expose que les colonnes nécessaires.

```
-- E1. Les serres de FR-BEAU : π code, surface_m2 (σ exploitation_id = 'FR-BEAU' (serre))
SELECT code, surface_m2 FROM serre WHERE exploitation_id = 'FR-BEAU' ORDER BY code;
-- Résultat : 2 lignes
```

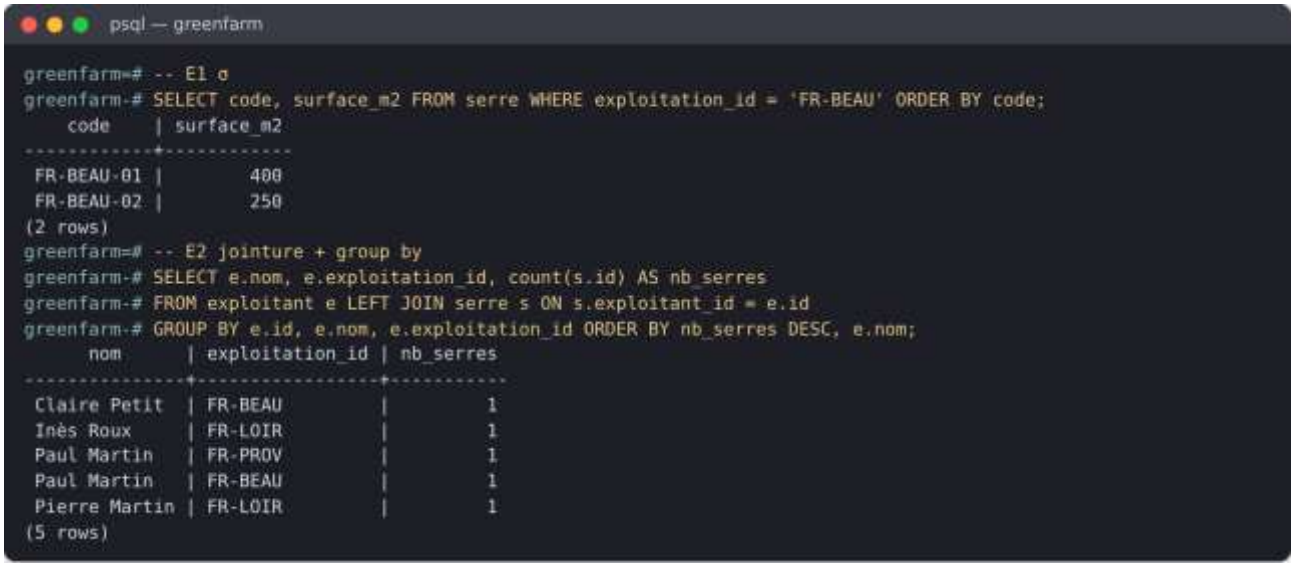
À vous · Q1

```
-- Q1. Les ruches du site SN-CASA, avec leur position
-- Algèbre : π ___ ( σ ___ ( ruche ) )
SELECT ... ;
-- Résultat : ___ lignes
```

Exemple GreenFarm · Relier deux tables et compter

La **jointure** ⋈ relie deux tables (`JOIN … ON`). Le `LEFT JOIN` garde aussi les exploitants sans serre ; `GROUP BY e.id` compte par personne, pas par nom.

```
-- E2. Chaque exploitant et son nombre de serres : exploitant ⋈ serre, puis comptage
SELECT e.nom, e.exploitation_id, count(s.id) AS nb_serres
FROM exploitant e LEFT JOIN serre s ON s.exploitant_id = e.id
GROUP BY e.id, e.nom, e.exploitation_id ORDER BY nb_serres DESC;
```



Les deux « Paul Martin » sont comptés séparément

À vous · Q2

Chaque apicultrice et son nombre de ruches. **Attendu :** une ligne par apicultrice ; les deux « Aminata Diop » séparées.

Exemple GreenFarm · Retrancher, réunir

La **différence** − (`EXCEPT`) retire d'un ensemble ce qui est dans un autre : les serres sans intervention. L'**union** ∪ (`UNION`) réunit deux résultats, sans doublon.

```
psql — greenfarm
greenfarm=# -- E3 différence
greenfarm-# SELECT code FROM serre
greenfarm-# WHERE id IN (SELECT id FROM serre EXCEPT SELECT serre_id FROM intervention) ORDER BY code;
    code
------------
 FR-BEAU-02
 FR-LOIR-02
(2 rows)
greenfarm=# -- E4 union
greenfarm-# SELECT exploitation_id FROM serre WHERE surface_m2 >= 400
greenfarm-# UNION
greenfarm-# SELECT exploitation_id FROM exploitant WHERE nom LIKE 'Inès%';
 exploitation_id
-----------------
 FR-PROV
 FR-BEAU
 FR-LOIR
(3 rows)
```

E3 : les serres jamais visitées ; E4 : une union

À vous · Q3 et Q4

- Q3 : les ruches jamais inspectées, avec `EXCEPT`. **Attendu :** au moins 2.
- Q4 : les sites qui ont au moins une alerte ouverte. **Attendu :** chaque site une seule fois (pensez à `DISTINCT`).

Exemple GreenFarm · Une vue qui n'expose que le nécessaire

Une **vue** est une requête enregistrée sous un nom. `v_serre_public` donne le dernier relevé de chaque serre, sans aucune donnée sur l'exploitant.

```
psql — greenfarm
greenfarm=# -- E5 projection sécurisée = vue
greenfarm-# CREATE OR REPLACE VIEW v_serre_public AS
greenfarm-# SELECT s.code, s.exploitation_id, r.temperature_c, r.horodatage
greenfarm-# FROM serre s
greenfarm-# JOIN LATERAL (SELECT temperature_c, horodatage FROM releve
greenfarm-#               WHERE serre_id = s.id ORDER BY horodatage DESC LIMIT 1) r ON true;
CREATE VIEW
greenfarm=# SELECT * FROM v_serre_public ORDER BY code;
    code     | exploitation_id | temperature_c |       horodatage
-------------+-----------------+---------------+------------------------
 FR-BEAU-01 | FR-BEAU         |          16.6 | 2027-03-15 00:00:00+01
 FR-BEAU-02 | FR-BEAU         |          17.0 | 2027-03-15 00:00:00+01
 FR-LOIR-01 | FR-LOIR         |          17.3 | 2027-03-15 00:00:00+01
 FR-LOIR-02 | FR-LOIR         |          16.6 | 2027-03-15 00:00:00+01
 FR-PROV-01 | FR-PROV         |          16.6 | 2027-03-15 00:00:00+01
(5 rows)
```

E5 : la vue v_serre_public

À vous · Q5, puis tout le fichier

1. Q5 : une vue `v_ruche_coordinateur` (code, site, dernière pesée), **sans** nom ni téléphone. **Attendu :** une ligne par ruche.
2. En fin de fichier, en commentaire : pourquoi une application ne doit-elle jamais faire `SELECT *` sur la table des apicultrices ?
3. **Enregistrez**, puis **dans le terminal** :

```
psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f requetes.sql
```

> **Résultat attendu :** les cinq résultats à la suite, sans `ERROR`.

Étape 3.2 · Un programme vulnérable, puis sa correction (45 min)

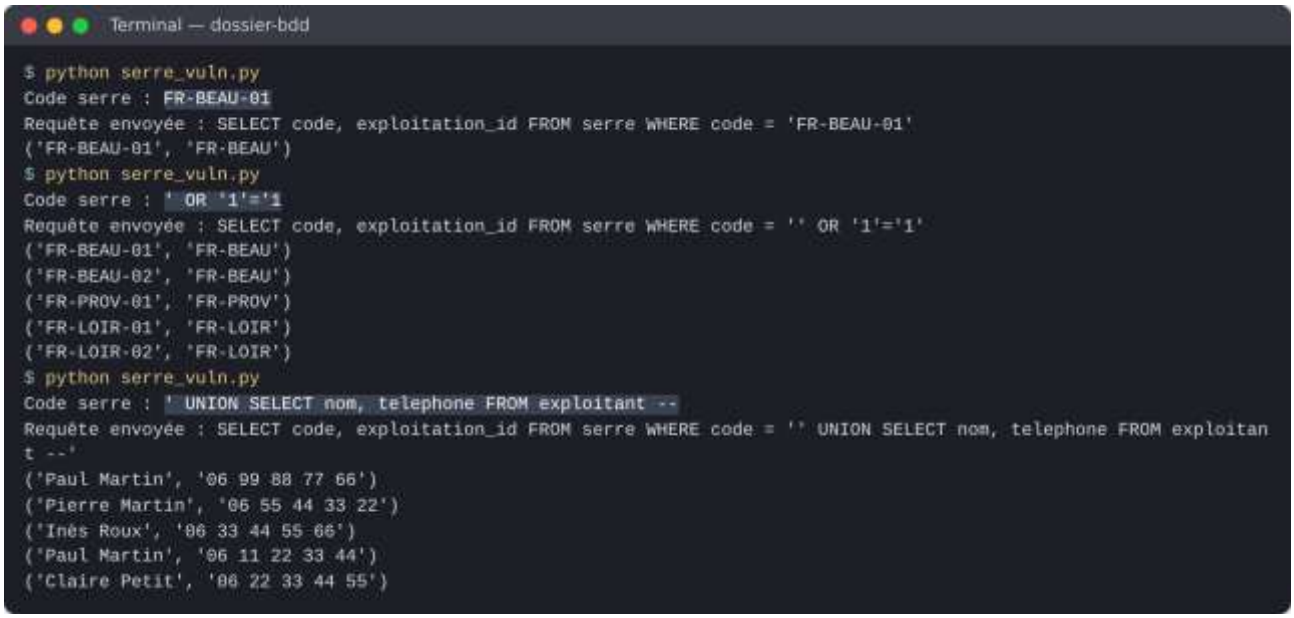
Jusqu'ici, **vous** écriviez les requêtes. Dans une application, c'est un **programme** qui les écrit, à partir de ce que tape l'utilisateur.

Exemple GreenFarm · Le programme vulnérable

`serre_vuln.py` colle la saisie dans le texte de la requête :

```
import psycopg
conn = psycopg.connect(dbname="greenfarm")
code = input("Code serre : ")
sql = "SELECT code, exploitation_id FROM serre WHERE code = '" + code + "'"
print("Requête envoyée :", sql)
for row in conn.execute(sql):
    print(row)
```

Lancé trois fois avec `python serre_vuln.py` : une saisie normale ; `' OR '1'='1`, qui rend la condition toujours vraie ; `' UNION SELECT nom, telephone FROM exploitant --`, qui ajoute une seconde requête. L'utilisateur a écrit du SQL : c'est l'**injection**.



1 ligne ; puis toutes les serres ; puis les noms et téléphones de tous les exploitants

À vous · Reproduire l'attaque

1. **Dans le terminal**, donnez au programme le mot de passe **sans l'écrire dans le code** : il le lira dans une variable du terminal.

```
# PowerShell
$env:PGUSER="postgres"; $env:PGPASSWORD="votre_mot_de_passe"; $env:PGHOST="localhost"
# Git Bash, macOS, Linux
export PGUSER=postgres PGPASSWORD=votre_mot_de_passe PGHOST=localhost
```

2. **Dans VS Code**, créez `query_vuln.py` : il demande un code de ruche et affiche le code et le site.
3. **Dans le même terminal**, lancez `python query_vuln.py` trois fois : un vrai code (attendu : 1 ligne) ; `' OR '1'='1` (attendu : toutes les ruches) ; `' UNION SELECT nom, telephone FROM apicultrice --` (attendu : les noms et téléphones des apicultrices).

Exemple GreenFarm · La correction

`serre.py` envoie la requête et la valeur **séparément** : la saisie n'est jamais lue comme du SQL. C'est la **requête paramétrée**, avec `%s` :

```
rows = conn.execute("SELECT code, exploitation_id FROM serre WHERE code = %s", (code,)).fetchall()
```



Avec %s, les deux attaques renvoient 0 ligne

À vous · Corriger et prouver

1. **Dans VS Code**, créez `query.py` avec `%s`. Relancez les trois saisies avec `python query.py`. **Résultat attendu :** 1 ligne, 0 ligne, 0 ligne.
2. En tête de `query.py`, notez vos résultats en commentaire :

```
# | Saisie                | query_vuln.py | query.py |
# | code normal           | 1 ligne       | 1 ligne  |
# | ' OR '1'='1           | ___           | ___      |
# | ' UNION SELECT ... -- | ___           | ___      |
# Pourquoi query.py résiste : ___
```

Enregistrez le lab. `git grep` cherche un mot dans les fichiers ajoutés : il ne doit rien trouver.

```
git add requetes.sql query_vuln.py query.py
git grep -i password
git commit -m "Lab 3 : requêtes et injection SQL"
```

Testez-vous · Lab 3

1. Quelle différence entre `JOIN` et `LEFT JOIN` pour compter les ruches de chaque apicultrice ?
2. Dans l'attaque `' UNION SELECT nom, telephone FROM apicultrice --`, à quoi servent l'apostrophe et les deux tirets ?
3. Pourquoi supprimer les apostrophes de la saisie ne suffit-il pas ?

4. Lab 4 · Dépendances fonctionnelles et clés candidates (1 h 30)

Pourquoi ce lab. Au lab 3, Q2 a montré que deux « Aminata Diop » ne sont pas la même personne : le nom ne **détermine** pas la personne. Ce lab rend ce raisonnement systématique : on cherche ce qui détermine quoi (les **dépendances fonctionnelles**), on en déduit ce qui peut servir de clé, et donc quelles règles `UNIQUE` poser.

Étape 4.1 · Tout mettre dans une seule table (15 min)

Pour voir les dépendances, on fait comme un export de tableur : une table où chaque mesure est recopiée avec tout son contexte.

Exemple GreenFarm

`plat.sql` joint les relevés, les serres, les exploitations, les exploitants et les groupements, et renomme les colonnes avec `AS` :

```
psql — greenfarm
greenfarm=# DROP TABLE IF EXISTS extrait_plat;
NOTICE:  table "extrait_plat" does not exist, skipping
DROP TABLE
greenfarm=# CREATE TABLE extrait_plat AS
greenfarm-# SELECT s.code        AS serre_code,
greenfarm-#        x.id          AS exploitation_id,
greenfarm-#        x.nom         AS exploitation_nom,
greenfarm-#        x.region,
greenfarm-#        e.nom         AS exploitant,
greenfarm-#        e.telephone,
greenfarm-#        g.nom         AS groupement,
greenfarm-#        r.horodatage,
greenfarm-#        r.temperature_c,
greenfarm-#        r.humidite_pct
greenfarm-# FROM releve r
greenfarm-# JOIN serre s        ON s.id = r.serre_id
greenfarm-# JOIN exploitation x ON x.id = s.exploitation_id
greenfarm-# JOIN exploitant e   ON e.id = s.exploitant_id
greenfarm-# JOIN adhesion a     ON a.exploitant_id = e.id
greenfarm-# JOIN groupement g   ON g.id = a.groupement_id;
SELECT 120
greenfarm=# SELECT count(*) FROM extrait_plat;
 count
-------
   120
(1 row)
```

Une ligne par relevé, avec tout le contexte recopié : 120 lignes

À vous

1. **Dans VS Code**, créez `plat.sql` : `DROP TABLE IF EXISTS extrait_plat;`, puis `CREATE TABLE extrait_plat AS SELECT …` qui joint les mesures, les ruches, les sites, les apicultrices, les adhésions et les coopératives, avec exactement ces 10 colonnes :

`ruche_code`, `site_id`, `site_nom`, `pays`, `apicultrice`, `telephone`, `cooperative`, `horodatage`, `poids_kg`, `temp_int_c`.

2. **Enregistrez**, puis **dans le terminal** :

```
psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f plat.sql
psql -U postgres -d queenbee -c "SELECT count(*) FROM extrait_plat"
```

Résultat attendu : 528 lignes, autant que de mesures.

Étape 4.2 · Trouver les dépendances et la clé, à la main (35 min)

Une **dépendance fonctionnelle** X → Y dit : « si je connais X, je connais Y ». C'est une règle métier, qu'on lit dans le sujet et dans le MCD.

Exemple GreenFarm · Les dépendances

DF	En français
`serre_code` → `exploitation_id`, `telephone`	une serre est dans une exploitation et a un exploitant
`exploitation_id` → `exploitation_nom`, `region`	une exploitation a un nom et une région
`telephone` → `exploitant`, `groupement`	un téléphone désigne un exploitant et son groupement
`serre_code`, `horodatage` → `temperature_c`, `humidite_pct`	un seul relevé par serre et par instant

À vous · Les dépendances

Dans VS Code, créez `dependances.md` et complétez :

```
# Dépendances fonctionnelles – Groupe ___

## Les DF de extrait_plat
| DF                              | En français |
|---------------------------------|-------------|
| site_id -> site_nom, pays       | un site a un seul nom et un seul pays |
| ___ -> ___                      | ___ |
| ___ -> ___                      | ___ |
| ruche_code, horodatage -> ___ | ___ |
```

Exemple GreenFarm · La fermeture et la clé

La **fermeture** X^+ est tout ce qu'on déduit de X, tour par tour, jusqu'à ce que rien ne s'ajoute. Si X^+ contient toutes les colonnes, X est une **clé** ; si on ne peut rien en retirer, c'est une **clé candidate**.

Tour	DF utilisée	{serre_code}⁺
0	aucune	{serre_code}
1	serre_code → exploitation_id, telephone	+ exploitation_id, telephone
2	exploitation_id → … ; telephone → …	+ exploitation_nom, region, exploitant, groupement
3	plus rien ne s'ajoute	7 colonnes sur 10 : **pas une clé**

{`serre_code`, `horodatage`}⁺ donne les 10 colonnes : c'est la clé candidate.

À vous · Trois fermetures et la clé

À la suite de `dependances.md`, faites le même tableau pour {`site_id`}, {`ruche_code`} et {`ruche_code`, `horodatage`}. Pour chacun, concluez : combien de colonnes sur 10, clé ou non. Puis écrivez la clé candidate, et pourquoi on ne peut rien en retirer.

Étape 4.3 · Confronter les dépendances aux données (20 min)

Une dépendance est une affirmation. Les données peuvent la contredire : il suffit d'une valeur de X associée à deux valeurs de Y. **Dans VS Code**, créez `contraintes.sql` ; testez chaque requête dans le **Query Tool** (*F5*) et collez le résultat en commentaire dessous.

Exemple GreenFarm · Une vraie dépendance, une fausse

La première requête teste `exploitation_id` → `exploitation_nom`, `region` : 0 ligne, rien ne la contredit. La seconde teste « le nom donne le téléphone » : Paul Martin a deux téléphones, elle est fausse.

```
psql — greenfarm

greenfarm=# -- DF
greenfarm-# SELECT exploitation_id, count(DISTINCT (exploitation_nom, region)) AS versions
greenfarm-# FROM extrait_plat GROUP BY exploitation_id HAVING count(DISTINCT (exploitation_nom, region)) > 1;
 exploitation_id | versions
-----------------+----------
(0 rows)
greenfarm=# SELECT exploitant, count(DISTINCT telephone) AS nb_telephones
greenfarm-# FROM extrait_plat GROUP BY exploitant HAVING count(DISTINCT telephone) > 1;
  exploitant | nb_telephones
-------------+---------------
 Paul Martin |             2
(1 row)
```

0 ligne pour la vraie DF ; deux téléphones pour « Paul Martin »

À vous

1. Une requête par dépendance de votre tableau, sur le modèle `SELECT X FROM extrait_plat GROUP BY X HAVING count(DISTINCT Y) > 1;`. **Attendu :** 0 ligne pour chacune.
2. La fausse dépendance `apicultrice` → `telephone`. **Attendu :** « Aminata Diop », avec 2 téléphones.
3. Dans `dependances.md` : que se passerait-il si l'application identifiait les apicultrices par leur nom ?

Étape 4.4 · Rendre les logins uniques (20 min)

Le login devrait être une clé candidate de la table des comptes : un login, une personne. Votre jeu d'essai a fabriqué trois `a.diop`. Les commandes suivantes modifient les données et dépendent l'une de l'autre : écrivez-les à la suite de `contraintes.sql` et exécutez-les **une par une, dans l'ordre**, dans le Query Tool.

Exemple GreenFarm · Les doublons bloquent la contrainte

```
psql — greenfarm

greenfarm=# -- logins
greenfarm-# SELECT c.login, e.nom, e.exploitation_id FROM compte c LEFT JOIN exploitant e ON e.id = c.exploitant_id
greenfarm-# WHERE c.login IN (SELECT login FROM compte GROUP BY login HAVING count(*) > 1) ORDER BY c.login, e.nom;
  login   |      nom      | exploitation_id
----------+---------------+-----------------
 p.martin | Paul Martin   | FR-BEAU
 p.martin | Paul Martin   | FR-PROV
 p.martin | Pierre Martin | FR-LOIR
(3 rows)
greenfarm=# ALTER TABLE compte ADD CONSTRAINT compte_login_unique UNIQUE (login);
ERROR:  could not create unique index "compte_login_unique"
DETAIL:  Key (login)=(p.martin) is duplicated.
```

Trois comptes p.martin : la contrainte UNIQUE est refusée

À vous · Commandes 1 et 2

1. Afficher les logins en double. **Attendu :** `a.diop`, trois fois.
2. `ALTER TABLE utilisateur ADD CONSTRAINT utilisateur_login_unique UNIQUE (login);` **Attendu :** refus.

Exemple GreenFarm · Corriger, puis poser la contrainte

```
psql — greenfarm
greenfarm=# UPDATE compte c SET login = 'pierre.martin' FROM exploitant e WHERE e.id = c.exploitant_id AND e.nom =
'Pierre Martin';
UPDATE 1
greenfarm=# UPDATE compte c SET login = 'paul.martin.prov' FROM exploitant e WHERE e.id = c.exploitant_id AND e.tel
ephone = '06 99 88 77 66';
UPDATE 1
greenfarm=# ALTER TABLE compte ADD CONSTRAINT compte_login_unique UNIQUE (login);
ALTER TABLE
```

Deux logins corrigés, la contrainte passe

À vous · Commandes 3 et 4

3. Des `UPDATE` qui donnent un login différent à chaque personne.
4. La même contrainte qu'en 2. **Attendu :** acceptée.

Exemple GreenFarm · Le piège des majuscules

`P.Martin` passe la contrainte, car la majuscule rend la valeur différente ; mais à la connexion, beaucoup d'applications ignorent la casse. Un index unique sur `lower(login)` ferme la porte.

```
psql — greenfarm
greenfarm=# INSERT INTO compte (login, role) VALUES ('P.Martin', 'admin');
INSERT 0 1
greenfarm=# DELETE FROM compte WHERE login = 'P.Martin';
DELETE 1
greenfarm=# CREATE UNIQUE INDEX compte_login_ci ON compte (lower(login));
CREATE INDEX
greenfarm=# INSERT INTO compte (login, role) VALUES ('P.Martin', 'admin');
ERROR:  duplicate key value violates unique constraint "compte_login_ci"
DETAIL:  Key (lower(login))=(p.martin) already exists.
```

P.Martin est refusé par l'index unique sur lower(login)

À vous · Commandes 5 et 6

5. Insérer un compte `A.Diop`. **Attendu :** accepté. Supprimez-le.
6. `CREATE UNIQUE INDEX utilisateur_login_ci ON utilisateur (lower(login));`, puis de nouveau `A.Diop`. **Attendu :** refus.

En fin de fichier, en commentaire : quelles lettres STRIDE un login partagé permet-il ?

Enregistrez le lab :

```
git add plat.sql dependances.md contraintes.sql
git commit -m "Lab 4 : dépendances fonctionnelles et logins"
```

Testez-vous · Lab 4

1. Que veut dire `site_id` → `pays` ?
2. Pourquoi {`ruche_code`} n'est-il pas une clé de `extrait_plat` ?
3. Pourquoi « 0 ligne » ne prouve-t-il pas qu'une DF est vraie ?

5. Lab 5 · Formes normales (1 h 30)

Pourquoi ce lab. `extrait_plat` recopie le pays d'un site sur des centaines de lignes. Que se passe-t-il si une seule copie change ? Vous allez le provoquer, constater que la base se contredit, puis utiliser les dépendances du lab 4 pour la découper proprement et prouver que rien n'a été perdu.

Étape 5.1 · Provoquer les anomalies (30 min)

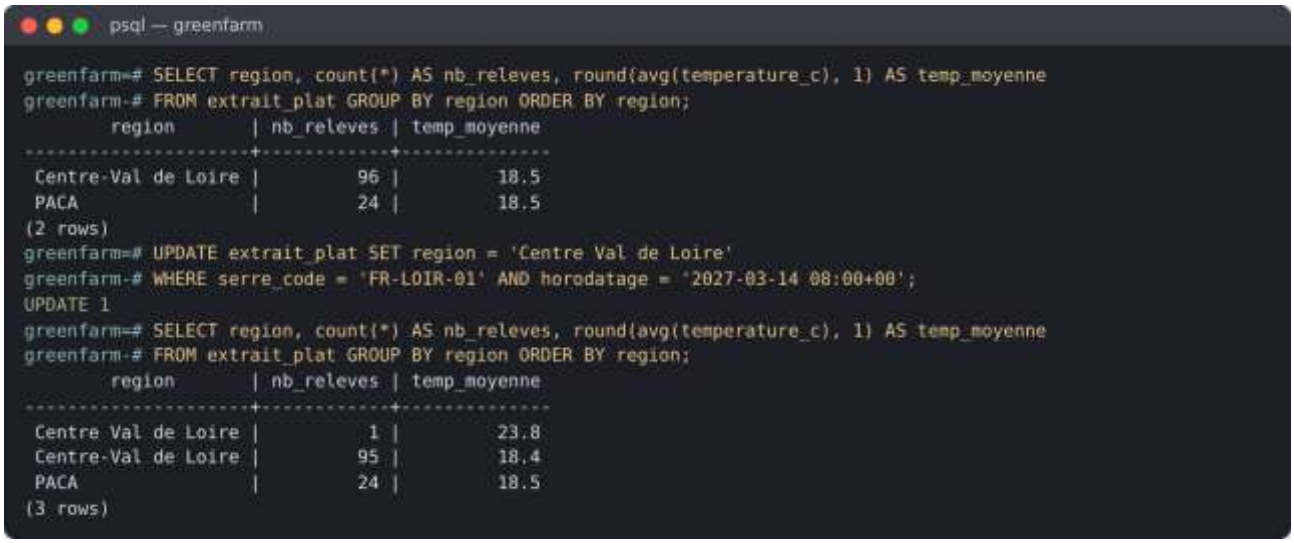
Un fait recopié sur plusieurs lignes crée trois **anomalies**. Avant de commencer, **dans le terminal**, relancez `plat.sql` pour repartir d'un extrait propre (sans la clé ajoutée au lab 4) :

```
psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f plat.sql
```

Dans VS Code, créez `anomalies.sql`. Chaque commande s'exécute **une par une** dans le Query Tool ; collez le résultat en commentaire.

Exemple GreenFarm · L'anomalie de mise à jour

Une région modifiée sur **une seule** ligne : l'enquête par région passe de 2 à 3 régions. Le même fait a maintenant deux valeurs.



« Centre Val de Loire » apparaît comme une troisième région

À vous · Mise à jour

1. L'enquête de référence : nombre de mesures et poids moyen par pays (`GROUP BY pays`). **Attendu :** 3 pays.
2. Sur une seule ligne, `Sénégal` devient `Senegal` ; sur une autre, changez le téléphone d'une apicultrice. Pour viser une seule ligne :

```
UPDATE extrait_plat SET pays = 'Senegal'
WHERE ruche_code = 'SN-KOLD-011'
  AND horodatage = (SELECT min(horodatage) FROM extrait_plat WHERE ruche_code = 'SN-KOLD-011');
```

3. Relancez l'enquête. **Attendu :** 4 pays.

Exemple GreenFarm · L'anomalie de suppression

Entre `BEGIN` et `ROLLBACK` (pour tout annuler ensuite), on supprime les relevés de la seule serre d'Inès Roux : Inès Roux disparaît de la base, alors qu'on voulait seulement effacer des relevés.

```
psql — greenfarm

greenfarm=# BEGIN;
BEGIN
greenfarm=*# DELETE FROM extrait_plat WHERE serre_code = 'FR-LOIR-02';
DELETE 24
greenfarm=*# SELECT count(*) FROM extrait_plat WHERE exploitant = 'Inès Roux';
 count
-------
     0
(1 row)
greenfarm=*# ROLLBACK;
ROLLBACK
```

Les relevés supprimés emportent l'exploitante

À vous · Suppression

Entre `BEGIN;` et `ROLLBACK;`, supprimez les lignes d'une apicultrice qui n'a qu'une ruche, puis cherchez son téléphone. **Attendu :** il a disparu.

Exemple GreenFarm · L'anomalie d'insertion

Avec la clé (`serre_code`, `horodatage`), on ne peut pas ajouter une nouvelle exploitation tant qu'elle n'a ni serre ni relevé.

```
psql — greenfarm

greenfarm=# ALTER TABLE extrait_plat ADD PRIMARY KEY (serre_code, horodatage);
ALTER TABLE
greenfarm=# INSERT INTO extrait_plat (exploitation_id, exploitation_nom, region) VALUES ('FR-BRET', 'Bretagne', 'Br
etagne');
ERROR:  null value in column "serre_code" of relation "extrait_plat" violates not-null constraint
DETAIL:  Failing row contains (null, FR-BRET, Bretagne, Bretagne, null, null, null, null, null, null).
```

Une exploitation sans serre est refusée

À vous · Insertion, puis le diagnostic

1. `ALTER TABLE extrait_plat ADD PRIMARY KEY (ruche_code, horodatage);`, puis essayez d'ajouter un nouveau site, sans ruche. **Attendu :** refus.
2. **Dans VS Code**, créez `normalisation.md` : pour chaque anomalie, la dépendance du lab 4 qui la cause. Puis classez vos dépendances avec ce tableau :

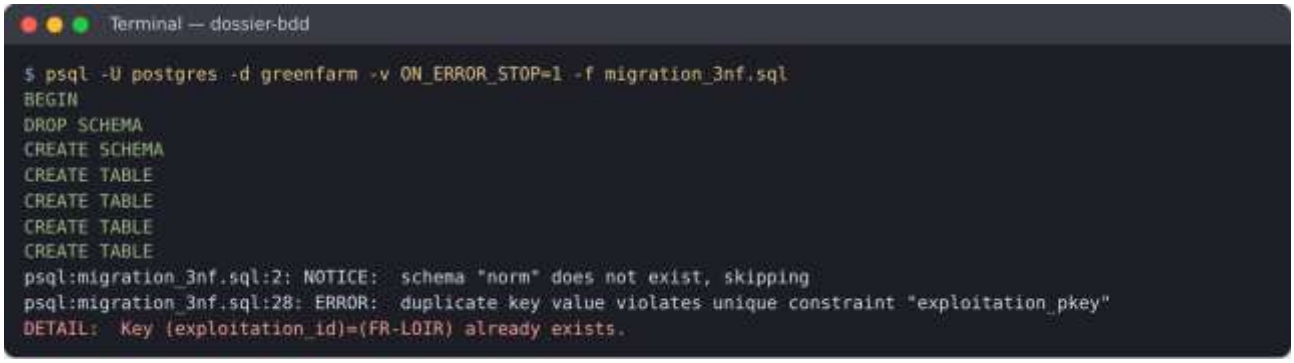
Forme normale	Règle	Vos dépendances qui la violent
1NF	chaque case contient une seule valeur	___
2NF	aucune colonne ne dépend d'une **partie** de la clé	___
3NF	aucune colonne ne dépend d'une colonne qui n'est pas la clé	___
BCNF	la partie gauche de toute dépendance est une clé	___

Étape 5.2 · Découper et migrer (35 min)

Le remède : **une table par déterminant** (la partie gauche d'une dépendance). Chaque fait n'est alors écrit qu'une fois.

Exemple GreenFarm · La migration échoue sur des données corrompues

`migration_3nf.sql` crée 4 tables dans un schéma à part, `norm`, et les remplit avec `INSERT … SELECT DISTINCT`. Lancé sur les données corrompues, il **échoue** : la clé primaire refuse qu'une exploitation ait deux régions. C'est le but : la base découpée ne peut plus se contredire.



FR-LOIR a deux régions : refus

À vous · Écrire la migration

1. **Dans VS Code**, créez `migration_3nf.sql` et complétez ce squelette avec vos dépendances :

```
BEGIN;
DROP SCHEMA IF EXISTS norm CASCADE;
CREATE SCHEMA norm;
CREATE TABLE norm.site        ( ___ );   -- déterminant : site_id
CREATE TABLE norm.apicultrice ( ___ );   -- déterminant : telephone
CREATE TABLE norm.ruche       ( ___ );   -- déterminant : ruche_code
CREATE TABLE norm.mesure      ( ___ );   -- clé : (ruche_code, horodatage)
INSERT INTO norm.site SELECT DISTINCT site_id, site_nom, pays FROM extrait_plat;
-- ... les trois autres INSERT
COMMIT;
```

2. **Enregistrez**, puis **dans le terminal** :

```
psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f migration_3nf.sql
```

Attendu : un refus (`duplicate key value violates unique constraint`).

Exemple GreenFarm · Nettoyer, puis relancer

On cherche les valeurs en conflit (`GROUP BY … HAVING count(DISTINCT …) > 1`), on garde la bonne, on corrige ; la migration passe.

```
psql — greenfarm

greenfarm=# SELECT exploitation_id, array_agg(DISTINCT region) FROM extrait_plat
greenfarm-# GROUP BY exploitation_id HAVING count(DISTINCT region) > 1;
 exploitation_id |                    array_agg
-----------------+-------------------------------------------------
 FR-LOIR         | {"Centre Val de Loire","Centre-Val de Loire"}
(1 row)
greenfarm=# UPDATE extrait_plat SET region = 'Centre-Val de Loire'
greenfarm-# WHERE region = 'Centre Val de Loire';
UPDATE 1
```

La région en conflit, puis sa correction

```
Terminal — dossier-bdd

$ psql -U postgres -d greenfarm -v ON_ERROR_STOP=1 -f migration_3nf.sql
BEGIN
DROP SCHEMA
CREATE SCHEMA
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 5
INSERT 0 5
INSERT 0 120
COMMIT
psql:migration_3nf.sql:2: NOTICE:  schema "norm" does not exist, skipping
```

3 exploitations, 5 exploitants, 5 serres, 120 relevés

À vous · Nettoyer, puis relancer

Dans le Query Tool, retrouvez **toutes** les incohérences (pays **et** téléphone), corrigez-les avec `UPDATE`, puis relancez la migration dans le terminal. **Attendu :** 4 sites, 8 apicultrices, 11 ruches, 528 mesures.

Étape 5.3 · Prouver que rien n'est perdu (25 min)

Le découpage est **sans perte** si la jointure des morceaux redonne exactement la table de départ.

Exemple GreenFarm · La preuve, puis les attaques rejouées

On recompose la table dans une vue, on la compare à l'original dans les deux sens : 0 ligne de différence. Puis on rejoue les attaques : une seconde région pour une exploitation est refusée ; une exploitation sans serre est acceptée.

```
psql — greenfarm
greenfarm=# CREATE VIEW norm.reconstruit AS
greenfarm-# SELECT s.serre_code, x.exploitation_id, x.exploitation_nom, x.region,
greenfarm-#        e.exploitant, e.telephone, e.groupement,
greenfarm-#        r.horodatage, r.temperature_c, r.humidite_pct
greenfarm-# FROM norm.releve r
greenfarm-# JOIN norm.serre s        USING (serre_code)
greenfarm-# JOIN norm.exploitation x USING (exploitation_id)
greenfarm-# JOIN norm.exploitant e   USING (telephone);
CREATE VIEW
greenfarm=# (SELECT * FROM extrait_plat EXCEPT SELECT * FROM norm.reconstruit)
greenfarm-# UNION ALL
greenfarm-# (SELECT * FROM norm.reconstruit EXCEPT SELECT * FROM extrait_plat);
 serre_code | exploitation_id | exploitation_nom | region | exploitant | telephone | groupement | horodatage | tempe
rature_c | humidite_pct
------------+-----------------+------------------+--------+------------+-----------+------------+------------+------
---------+--------------
(0 rows)
greenfarm=# INSERT INTO norm.exploitation VALUES ('FR-LOIR', 'Loiret', 'Centre Val de Loire');
ERROR:  duplicate key value violates unique constraint "exploitation_pkey"
DETAIL:  Key (exploitation_id)=(FR-LOIR) already exists.
greenfarm=# INSERT INTO norm.exploitation VALUES ('FR-BRET', 'Bretagne', 'Bretagne');
INSERT 0 1
```

La preuve sans perte (0 ligne) et les attaques rejouées

À vous

Dans le Query Tool :

1. Créez une vue `norm.reconstruit` qui joint vos quatre tables `norm` (mêmes colonnes, dans le même ordre, que `extrait_plat`), puis comparez :

```
(SELECT * FROM extrait_plat EXCEPT SELECT * FROM norm.reconstruit)
UNION ALL
(SELECT * FROM norm.reconstruit EXCEPT SELECT * FROM extrait_plat);
```

Attendu : 0 ligne.

2. Rejouez les attaques : un second pays pour un site existant (**attendu :** refus) ; un site sans ruche (**attendu :** accepté).
3. Relancez l'enquête par pays sur les tables `norm`. **Attendu :** de nouveau 3 pays.
4. Dans `normalisation.md`, vérifiez table par table que vos tables `norm` sont en **BCNF**, et concluez : « Une seule ligne modifiée a fait passer l'enquête de … à … pays. Une base non normalisée ne peut pas servir de preuve. »

Enregistrez le lab :

```
git add anomalies.sql migration_3nf.sql normalisation.md
git commit -m "Lab 5 : formes normales"
```

Testez-vous · Lab 5

1. Donnez un exemple de chacune des trois anomalies dans `extrait_plat`.
2. Pourquoi `ruche_code` → `site_id` viole-t-il la 2NF ?
3. Comment prouve-t-on en SQL qu'une décomposition est sans perte ?